

用户手册

概览

Tier0 是什么？

什么是 Tier0？

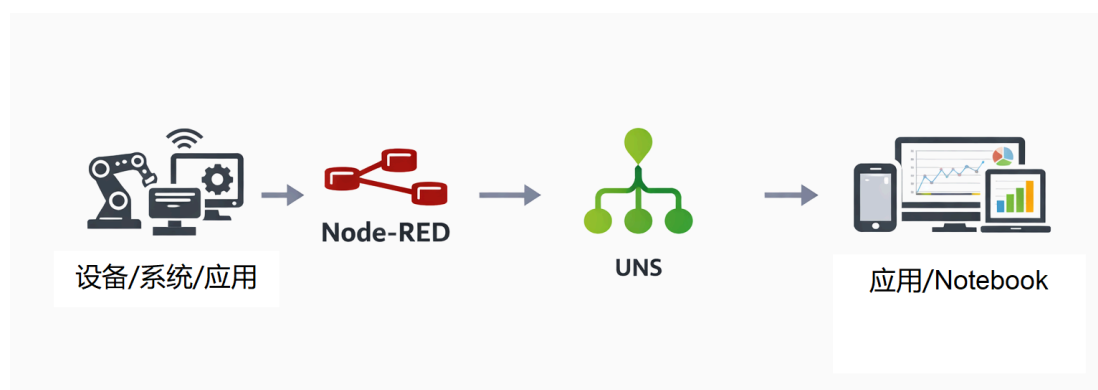
Tier0 是一个基于统一命名空间（UNS）的工业数据平台。

它通过 Node-RED 连接数据，将其组织为统一结构，并提供给上层应用和业务分析使用。

它如何工作？

Tier0 在一个统一的流程中对数据建模，处理数据：

- Node-RED 从设备和系统中连接并处理数据
- 数据被发布到统一命名空间（UNS），并在 Tier0 数据库中结构化存储
- 数据被仪表板、应用程序和分析工具消费



核心组件

统一命名空间（UNS）

为所有工业数据定义分层结构。

```
factory/workshopA/line1/cnc01/telemetry/spindleSpeed
```

工厂模型

模板 | 标签

请输入名称/别名/路径进行查找

Metric(1)

factory(1)

workshopA(1)

line1(1)

cnc01(1)

telemetry(1)

Metric(1)

spindleSpeed

factory / workshopA / line1 / cnc01 / telemetry / Metric / spindleSpeed

导入 | 导出

spindleSpeed

明细

定义

键	类型	长度	显示名称	备注
timeStamp	DATE TIME			
speed	INTEGER			
quality	LONG			

消息体

键	值	更新时间
---	---	------

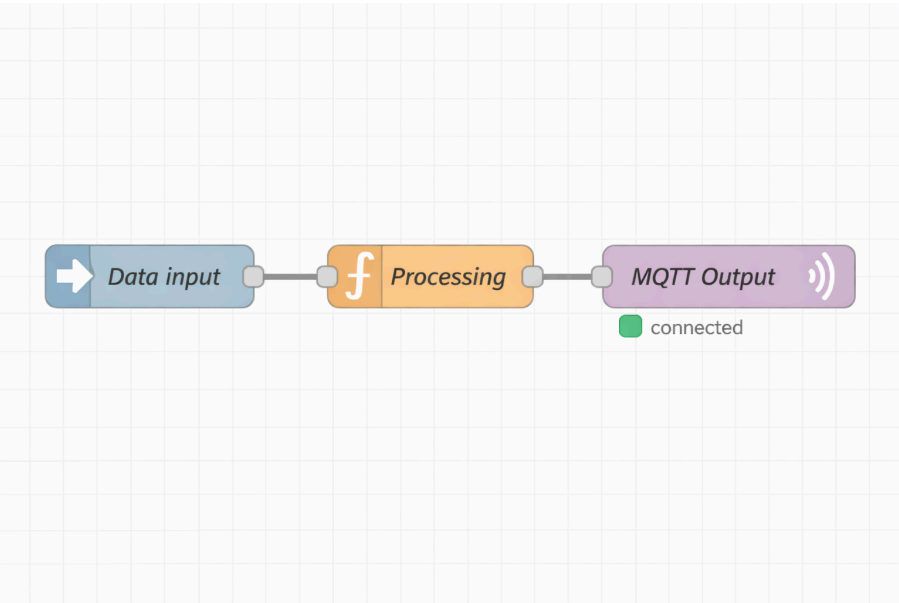
数据看板

创建

Node-RED 集成

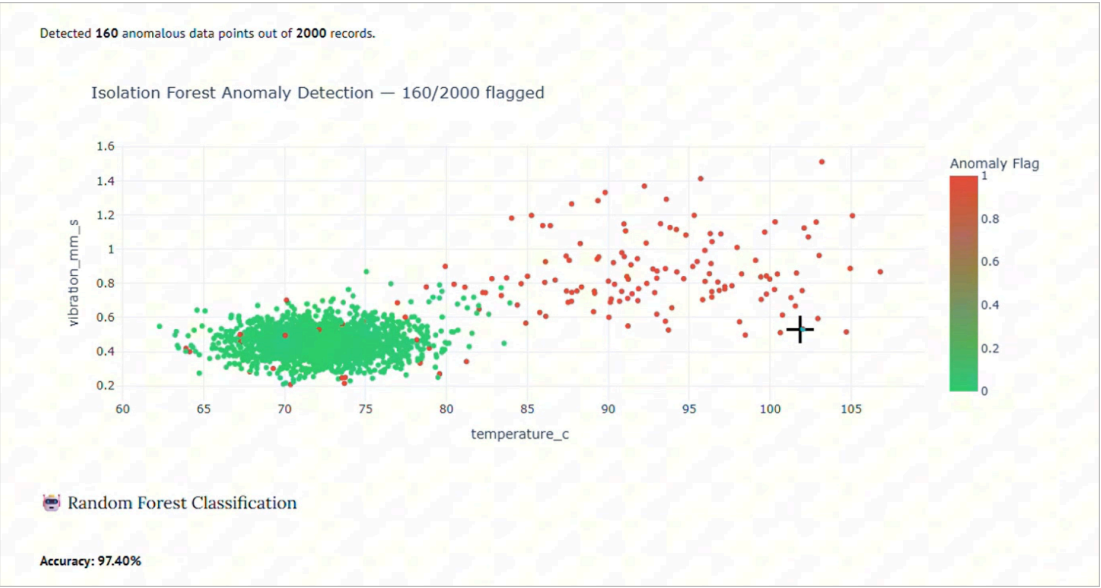
用于数据连接与处理的核心引擎。

- 连接设备、应用和系统
- 将数据格式化并发布到命名空间中



Python Notebook

使用 Python 在交互式 Notebook 中灵活分析数据。



Agentic 应用构建器

使用自然语言在云端创建工业应用，无需消耗任何本地资源。应用创建完成可打包本地安装使用。



首页

首页介绍



首页说明

序号	名称	说明
1	菜单栏	列出平台的功能菜单，可快速访问各个功能模块。
2	工具栏	平台常用操作，包括搜索、快速指南、任务及用户信息。
3	导入/导出	整合了命名空间、数据采集层、事件流转层、仪表盘的数据导入/导出功能。
		回到主页。
4	工厂模型	已在 UNS 中创建的数据模型树形视图
	模板	已在 UNS 中创建的数据模型模板树形视图
	标签	显示标签及每个标签下的关联模型
5		按类别展示数据模型：文本/模板/标签。
	/	搜索数据模型 / 刷新数据模型列表。
		根据数据层级，创建文件夹-文件结构的数据模型。
6	概述	数据源与内部 MQTT 代理的连接数及消息入/出量统计。
7	功能模块	列出了平台支持的功能模块。
8	MQTT 访问	内部 MQTT 代理的详细信息，可选择已有文件查看具体负载数据。

功能模块说明

模块分类	模块名称	说明
统一命名空间	数据建模	构建数据模型，为后续数据集成与使用奠定基础。
	数据连接	通过可视化流编程工具定义已创建模型的数据源。
	事件流程	通过可视化流编程工具自定义数据流，整合数据采集层与命名空间模块。
	数据看板	按特定设计在仪表盘中展示命名空间的数据。
工具集	容器管理	集成容器管理工具，管理容器。
	Notebook	集成交互式编程工具，方便使用Python脚本（例如：训练模型、执行SQL查询等）。
	高阶使用	提供集成应用的账号密码，方便深度探索。
系统管理	路由配置	自定义平台功能模块路由。
	用户管理	支持普通用户及角色管理。
	统一数据开放	提供不同类型的数据订阅和调用说明。包括API，MQTT及MCP。
	菜单配置	支持对所有现有功能模块的菜单进行自定义修改。

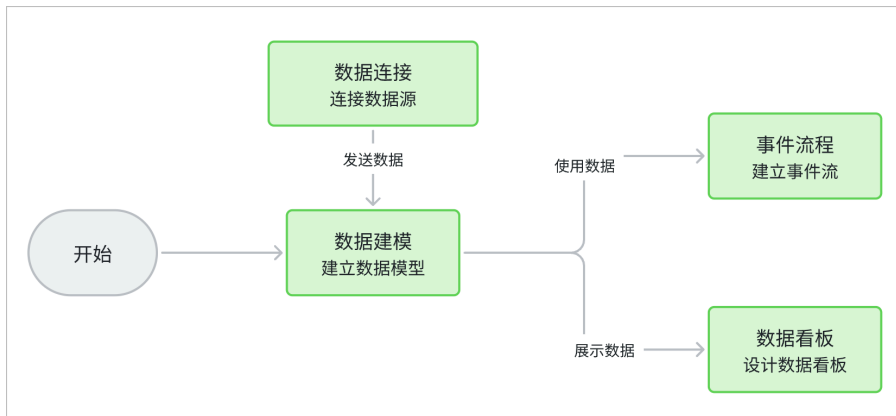
① 信息

- 私有化部署。具体部署要求及步骤，请参见[部署指南](#)。
- 通过自然语言构建的应用在云端生成，打包后在本地安装。

用户旅程

用户操作流程

按照图片所示顺序操作，即可完成数据流配置。



- 平台采用文件夹-文件结构构建数据模型，便于追踪和理解数据层级；
- 在数据连接中采集来自不同数据源的各类数据；
- 采集到平台的数据可在事件流程中进行处理；
- 处理后的数据可通过事件流程或数据看板模块，在可视化面板中展示，更直观地呈现数据价值；
- 采集到平台并经过处理的数据，第三方可通过 [OpenAPI](#) 或 MQTT 订阅获取。

① 备注

- 数据连接和事件流程基于Node-RED工具开发，详细使用说明请参考 [NodeRed 官方文档](#)；
- 数据看板模块基于Grafana平台开发，详细配置指南请参考 [Grafana 官方文档](#)。

入门指导

部署 Tier0

系统要求

配置类型	CPU	内存	硬盘/IOPS
基础配置	4核	8GB	100GB, 2000 IOPS (30%写入)
推荐配置	8核	16GB	1TB, 2000 IOPS (30%写入)

安装 Tier0

Linux 系统

1. 确保准备好以下系统和组件：

- Ubuntu Server 24.04
- Docker

组件	版本
Docker Engine - Community	27.4.0
Docker Buildx	v0.19.2
Docker Compose	v2.31.0
Containerd	1.7.24

① 备注

上述系统和组件均已测试通过，您也可以尝试其他版本。

2. 从 Github 克隆项目：

```
git clone https://github.com/FREEZONEX/Tier0-Edge.git
```

3. 进入项目内的 Tier0-Edge/deploy 文件夹:

```
cd Tier0-Edge/deploy
```

4. 打开 .env.default 文件, 按 i 键根据需要编辑配置项:

```
vi .env.default
```

配置项	说明
VOLUMES_PATH	项目数据的存储路径。
ENTRANCE_DOMAIN	Tier0 的访问地址, 通常是安装 Tier0 的服务器 IP 或域名。 ⚠ 注意 请勿使用 127.0.0.1 或 localhost, 否则登录和认证功能将无法正常运行。
LANGUAGE	Tier0 的显示语言, 支持中文 (zh-CN) 和英文 (en-US)。

```
# Deployed server specification
# for test
# supported: windows, linux
OS_PLATFORM_TYPE=linux

# Example: 1. 4c8g (4 CPU cores, 8GB RAM), 2. 8c16g (8 CPU cores, 16GB RAM) or higher
OS_RESOURCE_SPEC=1
OS_NAME=Tier0
VOLUMES_PATH=/volumes/supos/data

#(Your IP Address, do not to use 127.0.0.1 or localhost, otherwise the login and authentication functions will not work)
ENTRANCE_PROTOCOL=http
ENTRANCE_DOMAIN=192.168.1.100
ENTRANCE_PORT=8088
ENTRANCE_SSL_PORT=8443

# LANGUAGE setting, supported: en-US, zh-CN
LANGUAGE=en-US
```

5. 按 ESC 停止编辑, 保存文件并安装 Tier0:

```
:wq!
bash bin/install.sh
```

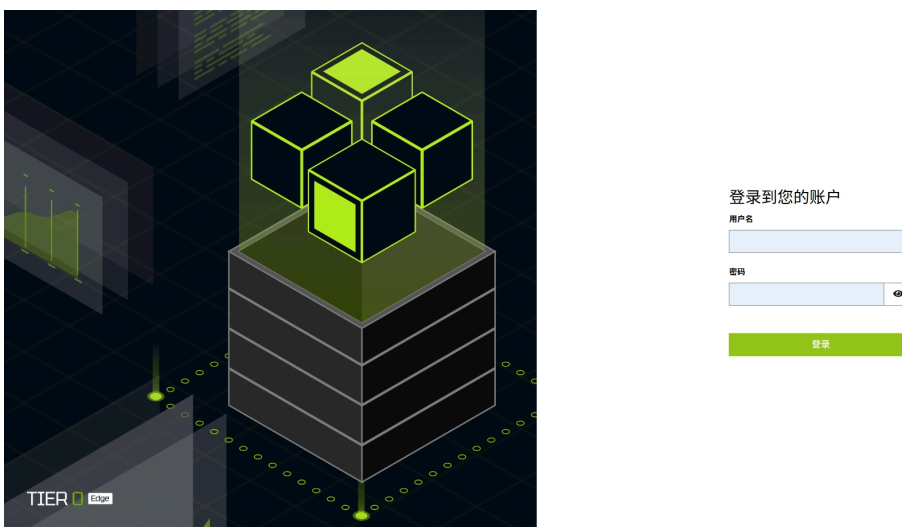
6. 选择是否保留输入的访问地址、是否使用默认设置, 然后等待安装完成。

```

Available options for ENTRANCE_DOMAIN:
0). Keep current: office.unibutton.com (default)
1). 192.168.31.167
2). 172.17.0.1
3). 172.18.0.1
Select option (0-3), or press Enter to keep current: 0
Keeping current ENTRANCE_DOMAIN: office.unibutton.com
Docker and Docker Compose meet the required versions.
Do you need to customize which services to install? [1] No, use defaults(default) [2] Yes 1
Info: success to generate /root/supOS-CE/bin/init/../../mount/postgresql/init-scripts/create_keycloak_database.sql
Info: success to generate kong_config.yml (auth enabled = true)
Info: loading npm cache complete.
goliath-websocket-datasource/
goliath-websocket-datasource/CHANGELOG.md
goliath-websocket-datasource/gpx_websocket_darwin_amd64
goliath-websocket-datasource/gpx_websocket_darwin_arm64
goliath-websocket-datasource/gpx_websocket_linux_amd64
goliath-websocket-datasource/gpx_websocket_linux_arm
goliath-websocket-datasource/gpx_websocket_linux_arm64
goliath-websocket-datasource/gpx_websocket_windows_amd64.exe
goliath-websocket-datasource/img/
goliath-websocket-datasource/img/assets/
goliath-websocket-datasource/img/assets/goliath-grafana-websockets-plugin-datasource.png
goliath-websocket-datasource/img/assets/grafana-websockets-graphing.png
goliath-websocket-datasource/img/assets/grafana-websockets-plugin-streaming.png
goliath-websocket-datasource/img/logo.svg
goliath-websocket-datasource/LICENSE
goliath-websocket-datasource/MANIFEST.txt
goliath-websocket-datasource/module.js
goliath-websocket-datasource/module.js.LICENSE.txt
goliath-websocket-datasource/module.js.map

```

7. 在浏览器中访问 Tier0 并登录。



Windows 系统

① 信息

- 确保已在 Windows 10 或 11 上安装最新版 Docker Desktop 和 Git。
- 启动 Docker。

1. 在 Git Bash 中从 Github 克隆项目：

```
git clone https://github.com/FREEZONEX/Tier0-Edge.git
```

2. 进入项目内的 Tier0-Edge/deploy 文件夹:

```
cd Tier0-Edge/deploy
```

3. 编辑 .env.default 文件中的以下属性:

✓ 提示

编辑 .env.default 文件前, 在 Git Bash 中执行 `ipconfig` 获取服务器 IP。

```
C:\Users\support>ipconfig /all

Windows IP Configuration

Host Name . . . . . : -10x64-T
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet0:

Connection-specific DNS Suffix . . :
Description . . . . . : Ethernet Adapter
Physical Address. . . . . : 00-5
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80:: (Preferred)
IPv4 Address. . . . . : 192.168. (Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : Friday, June 9, 2023 1:47:47 AM
Lease Expires . . . . . : Friday, June 9, 2023 5:07:47 AM
Default Gateway . . . . . : 192.168.
DHCP Server . . . . . : 192.168.
DHCPv6 IAID . . . . . : 503
DHCPv6 Client DUID. . . . . : 00-01-00-
DNS Servers . . . . . : 192.168.
NetBIOS over Tcpip. . . . . : Disabled
```

- OS_PLATFORM_TYPE: 安装 Tier0 的操作系统, 改为 windows
- VOLUMES_PATH: 项目数据的存储路径。
- ENTRANCE_DOMAIN/ENTRANCE_PORT: Tier0 的访问地址, 通常是安装 Tier0 的服务器 IP 或域名, 确保端口未被占用。

```
vi .env.default
```

① 备注

.env.default 文件默认隐藏, 直接输入命令即可打开。

```

MINGW64/d/supOS-CE
# Deployed server specification
# for test
# supported: windows, linux
OS_PLATFORM_TYPE=windows

# Example: 1. 4c8g (4 CPU cores, 8GB RAM), 2. 8c16g (8 CPU cores, 16GB RAM) or higher
OS_RESOURCE_SPEC=2
OS_NAME=supOS
VOLUMES_PATH=/c/Users/Free/volumes/supos/data

#(Your IP Address, do not to use 127.0.0.1 or localhost, otherwise the login and authentication functions will not work, )
ENTRANCE_PROTOCOL=http
ENTRANCE_DOMAIN=192
ENTRANCE_PORT=8

# LANGUAGE setting, supported: en-US, zh-CN
LANGUAGE=en-US

# Operating system version
OS_VERSION=V1.00.00.05-C

# MQTT TCP port
OS_MQTT_TCP_PORT=1883

# MQTT WebSocket TLS port
OS_MQTT_WEBSOCKET_TLS_PORT=8084

# Login path for the system
OS_LOGIN_PATH=/supos-login

# Enable or disable authentication (true/false)
OS_AUTH_ENABLE=true

# Type of large language model used
OS_LLM_TYPE=ollama

# Model and Key for oepnai (Enter your api key, can be empty)
OPENAI_API_MODEL=gpt-4o
OPENAI_API_KEY=

# Below are the environment variables required for using the built-in supOS MCP with the MCP Client (if applicable, can be empty)
# Langsmith API key
LANGSMITH_API_KEY=
# Agent service address (default: http://localhost:8123)
AGENT_DEPLOYMENT_URL=http://mcpclient:8123
# open API key for supOS, used for external access to the system
SUPOS_API_KEY=4174348a-9222-4e81-b33e-5d72d2fd7f1e
# The address of the MQTT server of supOS that needs to be accessed
SUPOS_MQTT_URL=tcp://emqx:1883/mqtt

#Variables from older versions, will be deleted for next iteration, please do not modify
MULTIPLE_TOPIC=false

#-----OAUTH CONFIG-----
OAUTH_REALM=supos
.env [unix] (10:22 20/05/2025)
".env" [unix] 69L, 1988B

```

4. 保存文件并开始安装 Tier0:

```

:wq!
bash bin/install.sh

```

5. 选择默认配置，等待安装完成。

```
MINGW64:/d/supOS-CE
Free@DESKTOP-JK7K19R MINGW64 /d/supOS-CE (main)
$ bash bin/startup.sh
Choose VOLUMES_PATH: (Press Enter for default: [/c/Users/Free/volumes/supos/data])
Choose IP address for ENTRANCE_DOMAIN (Press Enter for default: [192.168.1.1]):
Docker and Docker Compose meet the required versions.
Do you need to customize which services to install? [1] No, use defaults(default) [2] Yes 1
Info: success to generate /d/supOS-CE/bin/init/../../mount/postgresql/init-scripts/create_keycloak_database.s
ql
Info: success to generate kong_config.yml (auth enabled = true)
Info: loading npm cache complete.
golioth-websocket-datasource/
golioth-websocket-datasource/CHANGELOG.md
golioth-websocket-datasource/gpx_websocket_darwin_arm64
golioth-websocket-datasource/gpx_websocket_darwin_arm64
golioth-websocket-datasource/gpx_websocket_linux_arm64
golioth-websocket-datasource/gpx_websocket_linux_arm64
golioth-websocket-datasource/gpx_websocket_linux_arm64
golioth-websocket-datasource/gpx_websocket_windows_arm64.exe
golioth-websocket-datasource/img/
golioth-websocket-datasource/img/assets/
golioth-websocket-datasource/img/assets/golioth-grafana-websockets-plugin-datasource.png
golioth-websocket-datasource/img/assets/grafana-websockets-graphing.png
golioth-websocket-datasource/img/assets/grafana-websockets-plugin-streaming.png
golioth-websocket-datasource/img/logo.svg
golioth-websocket-datasource/LICENSE
golioth-websocket-datasource/MANIFEST.txt
golioth-websocket-datasource/module.js
golioth-websocket-datasource/module.js.LICENSE.txt
golioth-websocket-datasource/module.js.map
golioth-websocket-datasource/plugin.json
golioth-websocket-datasource/README.md
Info: success to create folder: /c/Users/Free/volumes/supos/data
time="2025-05-20T10:22:57+08:00" level=warning msg="D:\supOS-CE\docker-compose-8c16g.yml: the attribute 've
rsion' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 14/14
  ✓ Network supos_default_network Created 0.0s
  ✓ Container postgresql Healthy 62.0s
  ✓ Container eventflow Started 0.8s
  ✓ Container chat2db Started 0.8s
  ✓ Container konga Started 0.4s
  ✓ Container emqx Started 1.0s
  ✓ Container portainer Started 0.6s
  ✓ Container frontend Started 0.6s
  ✓ Container hasura Started 61.3s
  ✓ Container nodered Started 1.2s
  ✓ Container keycloak Healthy 81.9s
  ✓ Container grafana Started 61.4s
  ✓ Container backend Started 82.1s
  ✓ Container kong Started 82.3s
>> start to init nodered modules ...
added 55 packages in 4s
28 packages are looking for funding
  run 'npm fund' for details
npm warn deprecated node-opcua-client-crawler@2.127.1: true
```

6. 在浏览器中访问 Tier0 并登录。

卸载 Tier0

1. 进入路径 Tier0-Edge/deploy/bin。
2. 执行脚本 `uninstall.sh`:

```
bash uninstall.sh
```

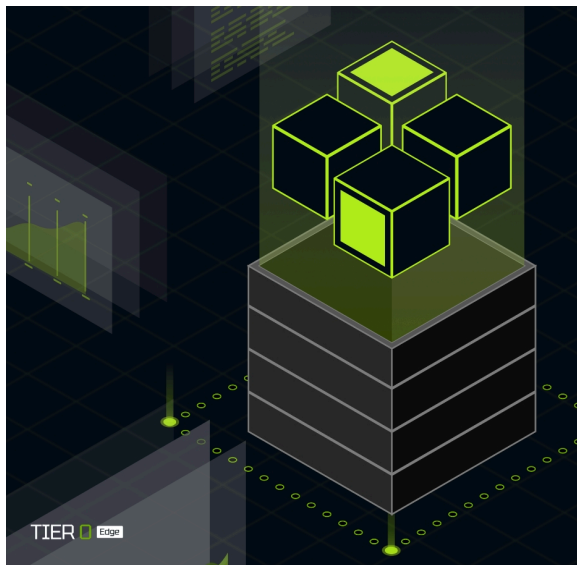
① 信息

- `clean-all.sh` 会清除所有项目数据并卸载Tier0。
- 镜像不会被删除。
- 如需重新安装 Tier0，请确保拉取所有最新版本的镜像（基于镜像标签）。

登录

1. 在浏览器中输入已设置的域名和端口，访问平台。

2. 在登录页面，输入用户名 `tier0` 和密码 `tier0`。



登录到您的账户

用户名

密码

登录

3. 单击 登录。

操作指南

建立数据模型

Tier0 采用统一命名空间（UNS）将数据组织成清晰的树形结构，模型中的每个节点都会按照树形图自动生成 MQTT 主题，方便获取实时数据。

补充参数说明

- 文件夹-更多设置

项目	说明
扩展属性	为文件夹添加扩展属性（例如：单位）。
模板	选择模板并继承其属性，详见 创建模板 。
生成模板	单击  为文件夹添加属性，同时可选择生成相同模板供后续使用。

- 文件-文件类型

类型	说明
状态	作为 API 接收系统状态数据。
操作	作为 API 执行增删改查（CRUD）命令。
指标	实时数据（例如：设备温度）。

- 文件-属性生成方式

属性生成方式	说明
自定义	自定义属性。
模板	从选中的模板继承属性。
自动解析	通过 JSON 文本生成相同结构的属性。

- 文件-可选行为

行为	说明
启用历史数据	为选中文件开启历史数据存储功能。

如何创建数据模型

基于简单的文件夹-文件结构，可将数据层级定义为树形视图。

手动创建模型

① 备注

factory/equipment/CNC 将作为示例，其中 `factory` 和 `equipment` 为文件夹，`CNC` 为文件。

1. 登录 Tier0，即进入数据建模。
2. 在工厂模型栏，单击  > 新建文件夹（例如：`factory`）。

新建文件夹

命名空间 ②

* 名称

▼ 更多设置

* 别名 ②

__02a670bbfed44188ba3d

显示名称

文件夹描述

扩展属性

⊕

模板

自定义

生成模板

☒

键 ②

名称

类型

▼

长度

显示名称(...)

备注(选填)

⊖

⊕

保存

- 3. 根据需求填写文件夹名称，别名等信息，单击保存。
- 4. 选中equipment，单击  > 新建文件，在其下添加文件（例如：CNC）。

新建文件

命名空间 ?

equipment/

* 名称

文件类型 ?

☒ 状态

☐ 操作

☐ 指标

属性生成方式 ?

☒ 自定义

☐ 模板

☐ 自动解析

键 ?

名称

类型

长度

显示名称...

备注(选填)

更多设置

* 别名 ?

__f0cb57cfb5b24622bb30

显示名称

文件描述

标签 ?

扩展属性

可选行为

☐ 启用历史数据 ?

保存

- 5. 输入文件相关信息，单击保存。

导入模型

- 1. 登录 Tier0，即进入数据建模。
- 2. 单击右上角的导入。



3. 导入 JSON 创建模型：

- 切换到UNS栏，直接输入 JSON 文本。

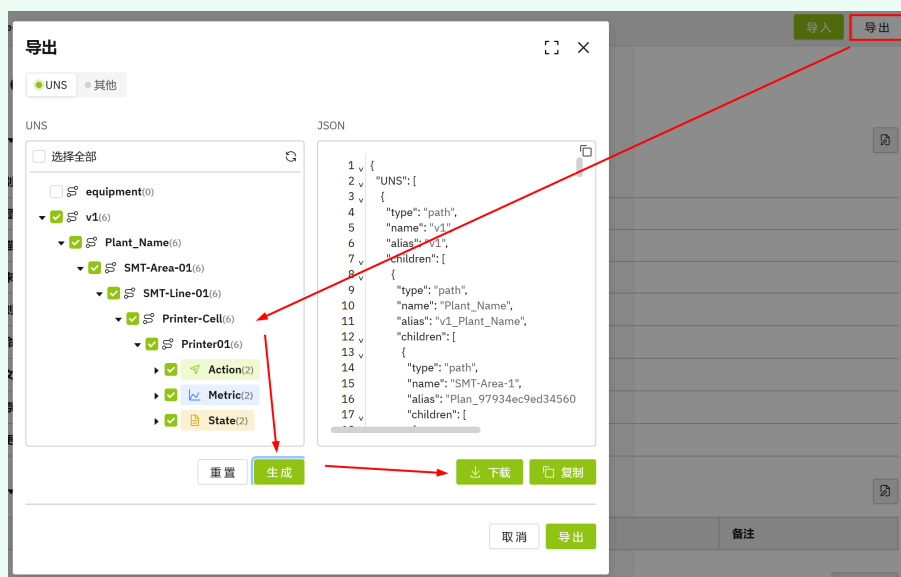


- 在UNS栏，单击下载模板，按模板规则填写模型内容后切换到全部栏上传。

```
{
  "notes": "type:folder|file,topicType:STATE|ACTION|METRIC,dataType:TEMPLATE_TYPE|TIME_SEQUENCE_TYPE|RELATION",
  "Label": [{"name": "debug"}, {"name": "dev"}],
  "Template": [],
  "UNS": [
    {
      "name": "v1",
      "type": "path",
      "children": [
        {
          "name": "Plant_Name",
          "type": "path",
          "children": [
            {
              "name": "SMT-Area-1",
              "type": "path",
              "children": [
                {
                  "name": "SMT-Line-1",
                  "type": "path",
                  "children": [
                    {
                      "name": "Printer-Cell",
                      "type": "path",
                      "children": [
                        {
                          "name": "Printer01",
                          "type": "path",
                          "children": [
                            {
                              "name": "State",
                              "type": "path",
                              "topicType": "STATE"
                            }
                          ]
                        }
                      ]
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

✓ 提示

可手动添加文件夹和文件后导出，将其作为导入示例参考。



4. 单击保存。

可选功能

创建模板

1. 在数据建模页面单击模板栏。
2. 单击 ，输入模板信息。

✓ 提示

单击来源可选择已添加的模型并继承其属性。

新建模板

* 名称

模板描述

键

来源

名称 类型 长度 显示名称(...) 备注(选填)

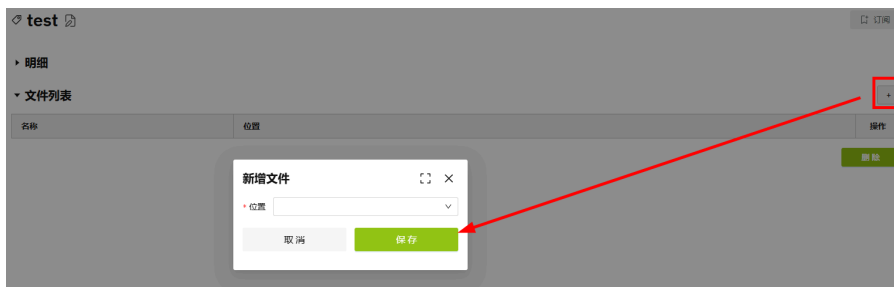
保存

3. 单击保存。

创建标签

标签用于对数据模型进行分类管理。

1. 在数据建模页面单击标签栏。
2. 单击 ，输入标签名称。
3. 单击保存，再单击  为标签添加文件。



4. 单击保存。

导出数据

Tier0 支持导出UNS模型以及其他数据，如数据连接，事件流程及数据看板。

1. 在数据建模页面单击右上角导出。

2. 单击其他栏，选择对应的数据类型。

导出

● UNS

● 其他

数据连接

请选择

事件流程

请选择

数据看板

请选择

取消

导出

3. 单击导出。

连接数据源

如何添加数据源

在数据建模下完成数据模型构建后，需为模型连接真实数据，以便后续处理和使用。

手动添加数据流

1. 登录平台，进入数据管理 > 数据连接。

数据连接

请输入名称或描述进行搜索

搜索

新建数据连接

名称	连接模板	描述	创建时间	创建人	操作
Equipment/Metric/CNC 运行中	node-red		2025/12/02 14:21:13	supos	
Equipment/状态/CNC 运行中	node-red		2025/12/02 14:12:01	supos	

总共 2 项 < 1 > 18 条/页

总共 2 项 < 1 > 18 条/页

序号	项目	说明
1	搜索	搜索现有数据流。
	新建数据连接	添加一条新的数据流。
2	数据流列表	显示数据流信息，单击流名称可进入数据流编辑页面。
3	显示方式	切换数据流的卡片式/列表式显示。
4	流操作	对数据流进行复制/编辑/删除操作。

- 单击**新建数据连接**，输入名称后单击保存。
- 单击刚创建的数据流，根据数据源类型拖拽对应节点（例如 **modbus** 节点）至画布。
- 拖拽一个 **MQTT输出** 节点，并完成配置：

- 添加内置代理：**emqx:1883**。

Edit mqtt out node > Add new mqtt-broker config node

Cancel Add

Properties

Name

Connection Security Messages

Server **emqx** Port **1883**

☒ Connect automatically

☐ Use TLS

Protocol MQTT V3.1.1

Client ID Leave blank for auto generated

Keep Alive 60

Session ☒ Use clean session

- 设置订阅主题为 **数据建模** 中的数据模型路径。

例如：数据模型结构为 **factory/equipment/cnc**，则主题设置为 **factory/equipment/cnc**。

Edit mqtt out node

Delete Cancel Done

Properties

Server emqx:1883

Topic factory/equipment/cnc

QoS Retain

Name Name

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

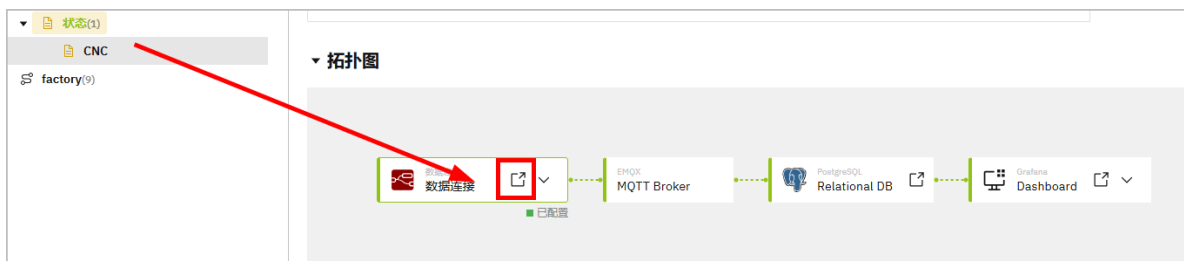
5. 单击右上角部署，触发数据流后，数据将发送至数据建模中对应的模型。

通过生成的流配置数据源

① 备注

创建文件时选择模拟数据，系统才会自动生成数据流。

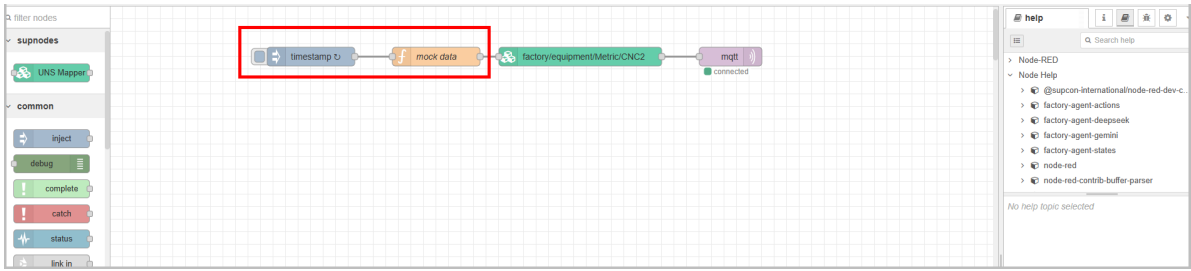
1. 登录平台，进入数据管理 > 数据建模，选择目标文件。
2. 向下滚动至拓扑图区域，单击数据连接右侧的图标，跳转至数据连接下自动生成的数据流页面。



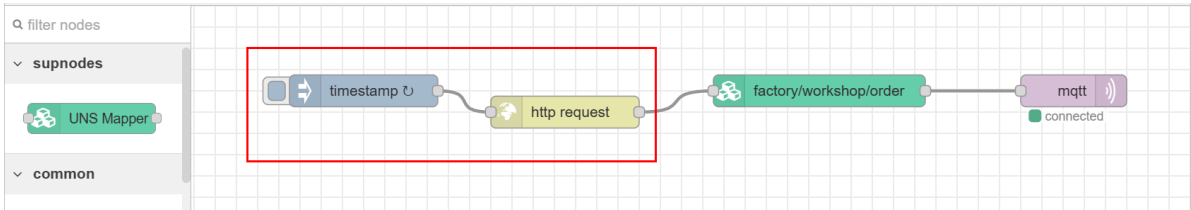
3. 修改数据源。

① 信息

确保源节点返回的数据格式为 JSON对象，目前平台仅支持解析该数据类型。



4. 删除模拟数据的函数节点，将源节点与后续流程连接。



5. 单击右上角部署，然后触发数据流。

6. 返回数据建模，检查数据是否已回传至该文件。

消息体		
键	值	更新时间
actualRuntime	1285	11:24:55 16/05/2025
plannedRuntime	1440	11:24:55 16/05/2025

常规数据源

连接 OPC UA

从OPCUA服务器读取数据

1. 在 数据连接 中，拖拽一个 `inject` 节点。
2. 双击该节点，输入 OPC UA 服务器中的 节点ID。

Edit inject node

Delete

Cancel

Done

Properties

Name

Name

msg. payload

=

milliseconds since epoch

x

msg. topic

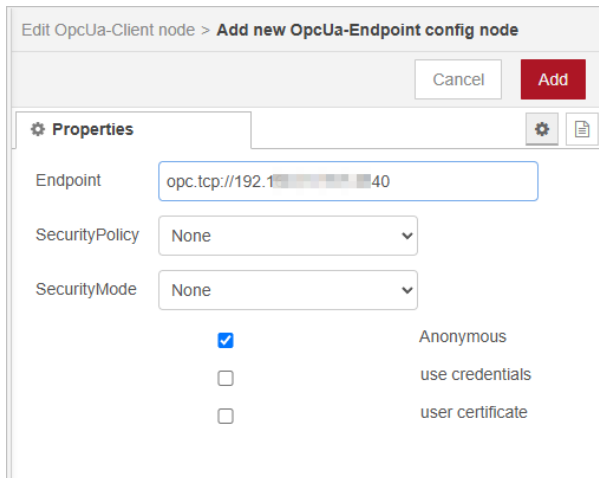
=

ns=2;i=2

x

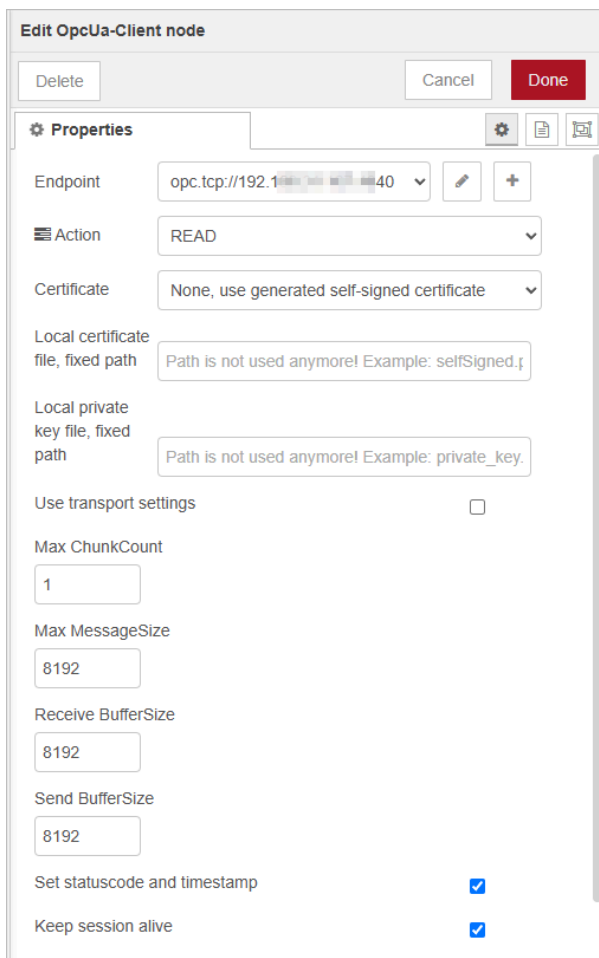
3. 拖拽一个 `OpcUa-Client` 节点，然后双击该节点。

4. 单击 Endpoint 右侧的  图标，输入你的 OPC UA 服务器信息。



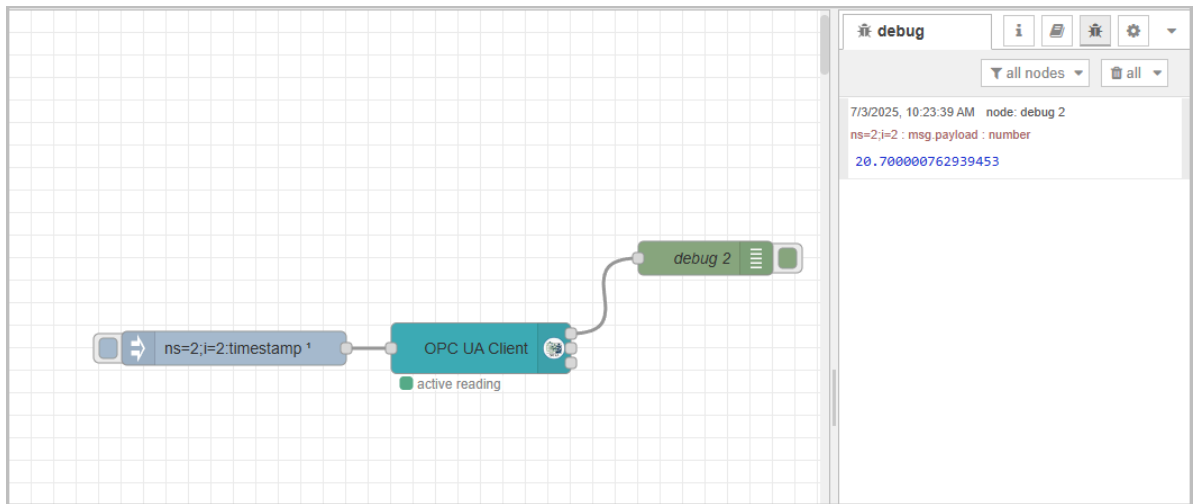
5. 单击 Add，将 Action 设置为 READ。

6. 其他参数保持默认配置，单击 Done。



7. 将 **inject** 节点与 **OpcUa-Client** 节点连接，最后在末端连接一个 **debug** 节点。

8. 单击右上角 Deploy，然后单击 **inject** 节点触发流程。



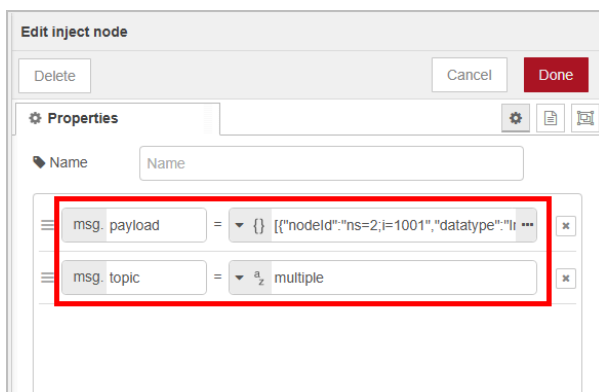
从OPCUA服务器订阅数据

1. 在 数据连接 中，拖拽一个 `inject` 节点。
2. 双击该节点，输入 OPC UA 服务器中的 节点ID。

- `msg.payload`:

```
[
  {
    "nodeId": "ns=2;i=1001",
    "datatype": "Int32"
  },
  {
    "nodeId": "ns=2;i=1002",
    "datatype": "Int32"
  }
]
```

- `msg.topic`: multiple



3. 拖拽一个 `OpcUa-Client` 节点，然后双击该节点。
4. 单击 Endpoint 右侧的 `+` 图标，输入你的 OPC UA 服务器信息。

Edit OpcUa-Client node > Add new OpcUa-Endpoint config node

Cancel Add

Properties

Endpoint

SecurityPolicy

SecurityMode

☒ Anonymous

☐ use credentials

☐ user certificate

5. 单击 Add，将 Action 设置为 SUBSCRIBE。

6. 设置订阅间隔，其他参数保持默认配置，单击 Done。

Edit OpcUa-Client node

Delete Cancel Done

Properties

Endpoint

Action

Interval

Certificate

Local certificate file, fixed path

Local private key file, fixed path

Use transport settings ☐

Max ChunkCount

Max MessageSize

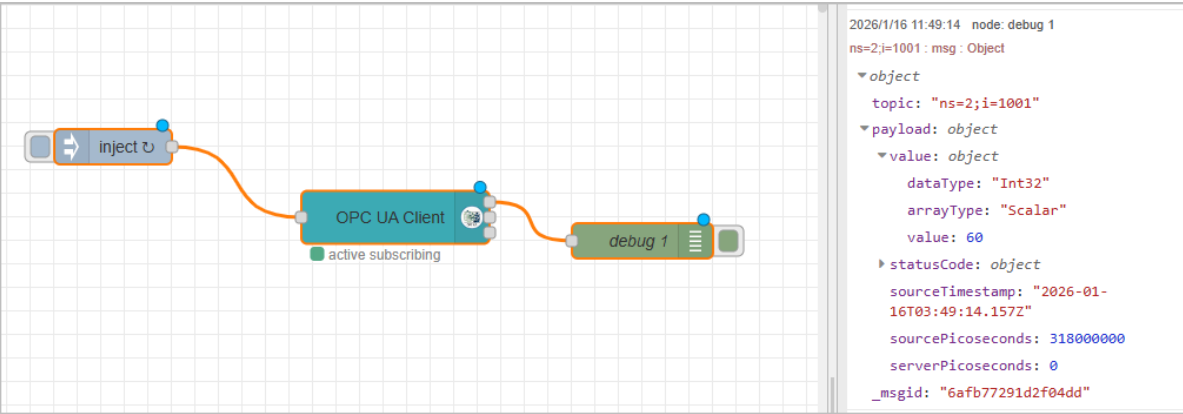
Receive BufferSize

Send BufferSize

Set statuscode and timestamp ☐

7. 将 inject 节点与 OpcUa-Client 节点连接，最后在末端连接一个 debug 节点。

8. 单击右上角 Deploy，然后单击 inject 节点触发流程。



连接 OPC DA

- 1. 在 数据连接 中，拖拽一个 inject 节点。
- 2. 拖拽一个 opcd-read 节点，然后双击该节点。
- 3. 单击 服务器 右侧的 + ，输入你的 OPC DA 服务器信息。

Edit opcd-read node > Add new opcd-server config node

Cancel

Add

⚙ Properties

⚙

📄

📄 Name

opcda

🌐 Address

192.168.31.142

🏠 Domain

"

👤 User Name

admin

🔑 Password

.....

📄 CIsId

7BC0CC8E-482C-47CA-ABDC-0FE7F9C6E729

🕒 Timeout

5000

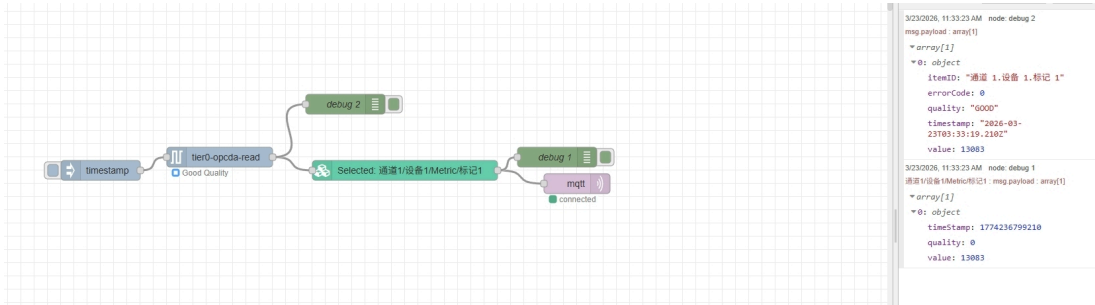
ms

🔍 Browse

📄 Export

参数	说明
地址 (Address)	OPC DA 服务器的 IP 地址。
域名 (Domain)	OPC DA 服务器的域名。
用户名/密码 (Username/Password)	登录 DA 服务器所用的用户名和密码。
应用标识 (CIsId)	所使用的 DA 连接服务的应用 ID（例如 KepServer）。

5. 单击 **Browse** 查看已连接的通道。
6. 单击 **Add**，在 **Items** 下添加所需的项目。
7. 将 **inject** 节点与 **opcda-read** 节点连接，最后在末端连接一个 **debug** 节点。
8. 单击右上角 **Deploy**，然后单击 **inject** 节点触发流程。



连接 Modbus

1. 在 **数据连接** 中，拖拽一个 **Modbus-Read** 节点。
2. 双击该节点，单击 **Server** 右侧的 **+**，输入你的 Modbus 服务器信息。

Edit Modbus-Read node > Add new modbus-client config node

Cancel Add

Properties

Settings Queues Optionals

Name modbus

Type TCP

Host 192.168.1.100

Port 502

TCP Type DEFAULT

Unit-Id 1

Timeout (ms) 1000

Reconnect on timeout ☒

Reconnect timeout (ms) 2000

3. 单击 **Add**，输入待连接数据的相关信息。

Delete
Cancel
Done

Edit Modbus-Read node

Properties

Settings
Optionals

Name
modbus

Topic
Topic

Unit-Id

FC
FC 3: Read Holding Registers

Address
0

Quantity
1

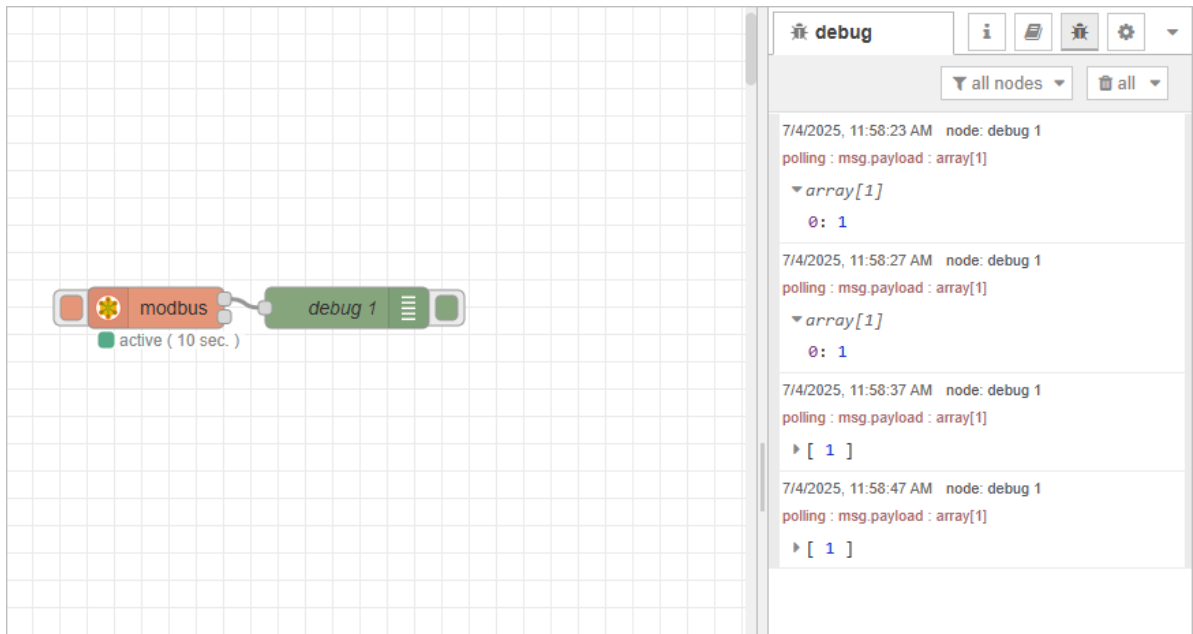
Poll Rate
10
second(s)

☐ Delay to activate input

Server
modbus

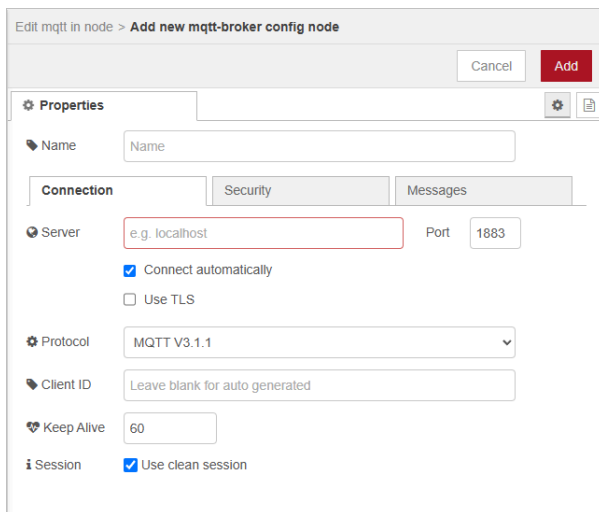
参数	说明
名称（Name）	节点的显示名称（仅用于可视化识别）。
主题（Topic）	可选参数，将传递至输出消息的主题字符串。
从站地址（Unit-Id）	目标设备的 Modbus 从站 ID（通常为 1 - 247）。
功能码（FC）	指定 Modbus 操作的功能码（例如：功能码 3 = 读取保持寄存器）。
起始地址（Address）	读取的起始寄存器地址（通常为零基地址）。
读取数量（Quantity）	连续读取的寄存器个数。
轮询频率（Poll Rate）	节点轮询 Modbus 设备的间隔时间（例如：每 10 秒一次）。
输入激活延迟（Delay to activate input）	可选参数，轮询输入信号激活前的延迟时间。
服务器（Server）	已配置的 Modbus 服务器引用（包含 IP、端口、协议等信息）。

- 单击 Done。
- 在末端连接一个 debug 节点。
- 单击右上角 Deploy，然后单击 Modbus-Read 节点触发流程。



连接 MQTT

1. 在 数据连接 中，拖拽一个 `mqtt in` 节点。
2. 双击该节点，单击 Server 右侧的 `+` 。



参数 (Parameter)	说明 (Description)
名称 (Name)	MQTT 代理配置的可选名称（仅用于内部参考）。
服务器 (Server)	MQTT 代理的主机名或 IP 地址（例如：localhost 或 broker.hivemq.com）。
端口 (Port)	用于连接 MQTT 代理的端口（非 TLS 连接默认值为 1883，TLS 连接默认值为 8883）。
自动连接 (Connect automatically)	启用后，Node-RED 会在流程启动时自动尝试连接代理。
使用 TLS (Use TLS)	若代理要求安全的 TLS/SSL 连接，需启用此选项。
协议 (Protocol)	用于连接的 MQTT 协议版本（例如：MQTT v3.1.1 或 v5.0）。
客户端 ID (Client ID)	可选的客户端 ID。若留空，系统会自动生成唯一 ID，且每个客户端的 ID 必须唯一。
保活时间 (Keep Alive)	与代理通信的最大间隔时间（单位：秒），用于检测断开的连接。
使用干净会话 (Use clean session)	启用后，不会与代理保持持久会话；未启用时，会话状态（如订阅信息）会被保留。
用户名 (Username)（安全标签页）	若需要，填写用于 MQTT 代理身份验证的用户名。
密码 (Password)（安全标签页）	与用户名对应的代理身份验证密码，会被安全存储。

3. 单击 Add，输入需要订阅的主题。

4. 单击 Done。

Edit mqtt in node

Delete Cancel Done

Properties

Server: emqx

Action: Subscribe to single topic

Topic: factory/RCS/AGV

QoS: 2

Output: auto-detect (parsed JSON object, string or buff)

Name: Name

5. 在末端连接一个 `debug` 节点。
6. 单击右上角 部署 (Deploy)，当该主题接收到数据时，流程会被触发。

factory/RCS/AGV

debug 1

connected

debug

all nodes

all

7/4/2025, 2:24:45 PM node: debug 1
factory/RCS/AGV : msg.payload : Object

object

no: 66
position: "3fcLKwHDAvthuZLEwW8V"
state: "DqGG213IuzMDT73GnVS3"
battery: "62.91"

7/4/2025, 2:24:55 PM node: debug 1
factory/RCS/AGV : msg.payload : Object

{ no: 60, position: "6CcOvOZfJM8zaCCJA15e", state: "zmP15enJhhK064mK16wV", battery: "63.99" }

连接文件

① 信息

在数据连接中处理文件前，需将文件放入对应的容器中。

```
scp C:\Users\you\Desktop\myfile.txt username@server IP:/home/username/  
//将文件从本地复制到服务器  
docker cp /root/config.json <容器名称或ID>:/app/config.json  
//将文件从服务器复制到容器
```

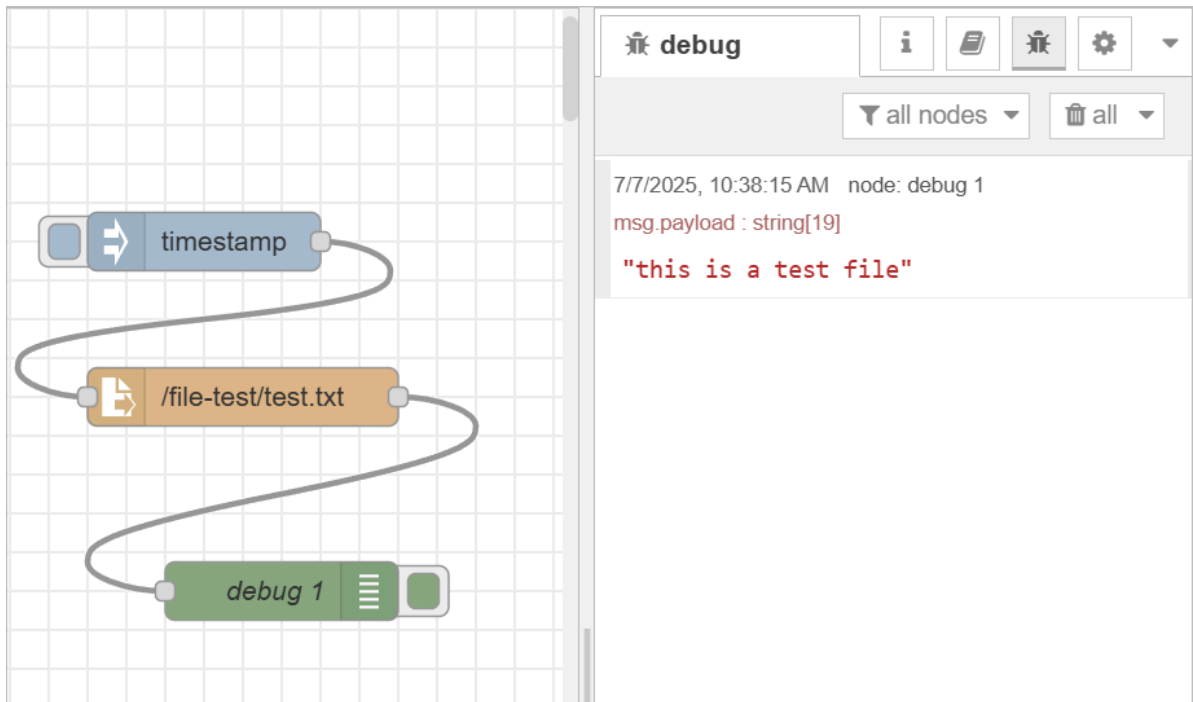
✓ 常用Docker命令

命令	说明
<code>docker ps</code>	列出所有正在运行的Docker容器。
<code>docker ps grep [container]</code>	根据指定的容器信息筛选正在运行的容器。
<code>docker exec -it [container] /bin/sh</code>	通过 <code>/bin/sh</code> 在运行中的容器内启动一个 shell会话。
<code>mkdir [directory]</code>	在容器内创建一个指定名称的新目录。

连接文本文件

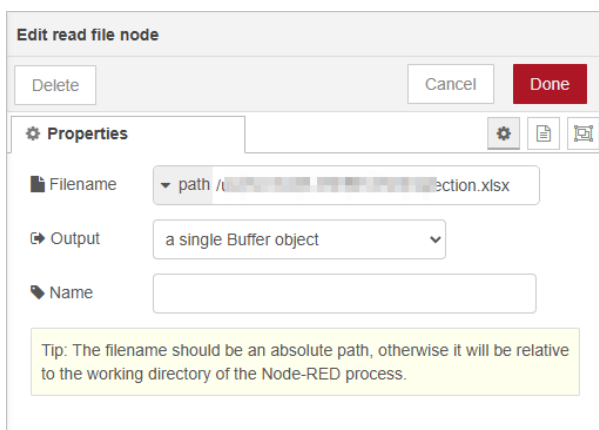
1. 在 数据连接 中，拖拽一个 `inject` 节点。
2. 拖拽一个 `read file` 节点，然后双击该节点。
3. 输入文件路径和输出配置。

4. 单击 完成，然后在末端连接一个 `debug` 节点。
5. 单击右上角 部署，接着单击 `inject` 节点触发流程。

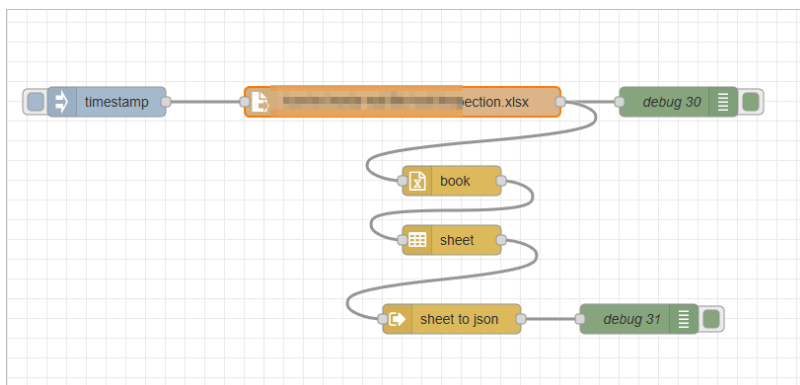


连接Excel文件

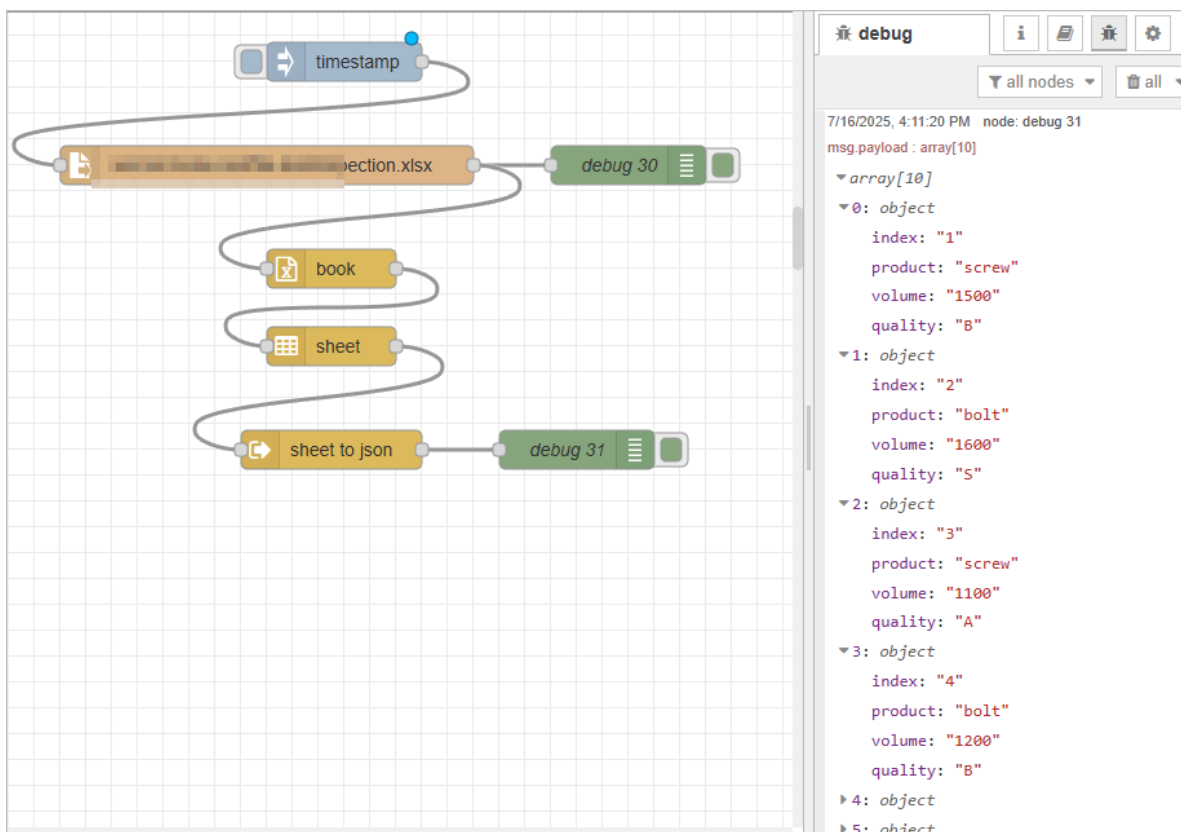
1. 在 数据连接 中，拖拽一个 **inject** 节点。
2. 拖拽一个 **read file** 节点，然后双击该节点。
3. 输入文件路径和输出配置。



4. 单击 **Done**，然后按顺序连接 **book**、**sheet** 和 **sheet to json** 节点，将表格数据转换为JSON格式。



5. 单击右上角 Deploy，接着单击 **inject** 节点触发流程。



连接 RestAPI

1. 在 数据连接 中，拖拽一个 **inject** 节点。
2. 拖拽一个 **http request** 节点，然后双击该节点。
3. 输入API相关信息。

✓ 提示

需将 Return 设置为 a parsed JSON object，以便平台的数据建模模块解析。

Edit http request node

Delete Cancel Done

Properties

Method GET

URL http://

Payload Ignore

☐ Enable secure (SSL/TLS) connection

☐ Use authentication

☐ Enable connection keep-alive

☐ Use proxy

☐ Only send non-2xx responses to Catch node

☐ Disable strict HTTP parsing

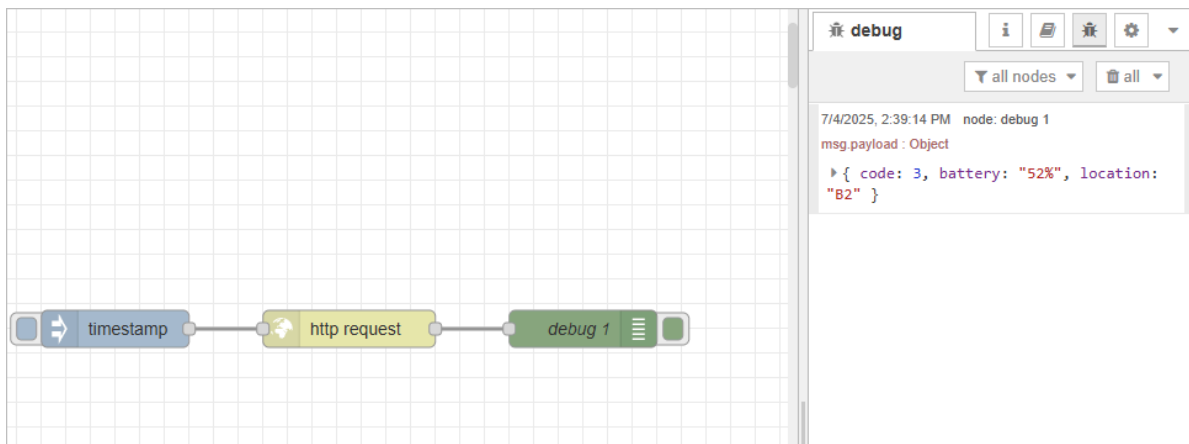
Return a parsed JSON object

Tip: If the JSON parse fails the fetched string is returned as-is.

Headers

+ add

5. 单击 完成，然后在末端连接一个 **debug** 节点。
6. 单击右上角 **Deploy**，接着单击 **inject** 节点触发流程。



配置事件流程

如何配置事件流程

1. 登录Tier0，然后选择 数据管理 > 事件流程。

事件流程					Q 请输入名称/描述进行搜索		1	搜索	+ 新建事件流程
名称	连接模板	描述	创建时间	创建人	2	3	4	操作	三
test	event-flow		2025/11/25 17:48:15	supos					
总共 1 项 < 1 > 18 条/页									

序号	项目	说明
1	搜索	搜索现有数据流。
	新建事件流程	添加一条新的数据流。
2	流列表	显示数据流信息，单击流名称可开始编辑数据流。
3	显示方式	切换数据流的卡片式/列表式显示。
4	流操作	对数据流进行复制/编辑/删除操作。

2. 单击 新建事件流程，输入名称后单击 保存。
3. 单击刚创建的数据流，构建数据流以分析数据。

常规数据处理

整合多源数据

示例

平台已连接3个数据源，需将这些数据源的数据合并后用于后续处理。

```
{
  "workshop": {
    "furnace1": {
      "temp": 85
    },
    "furnace2": {
      "humidity": 64
    },
    "furnace3": {
      "pressure": 80
    }
  }
}
```

需使用的节点

本示例中，我们使用 `join` 节点来整合之前接收的多条消息。

Edit join node

Delete

Cancel

Done

⚙ Properties

⚙

📄

🖨

📌 Name

Name

Mode

manual

▼

Combine each

▼ msg. payload

to create

a merged Object

▼

☐ Use existing msg.parts property

Send the message:

• After a number of message parts

3

☒ and every subsequent message.

• After a timeout following the first message

seconds

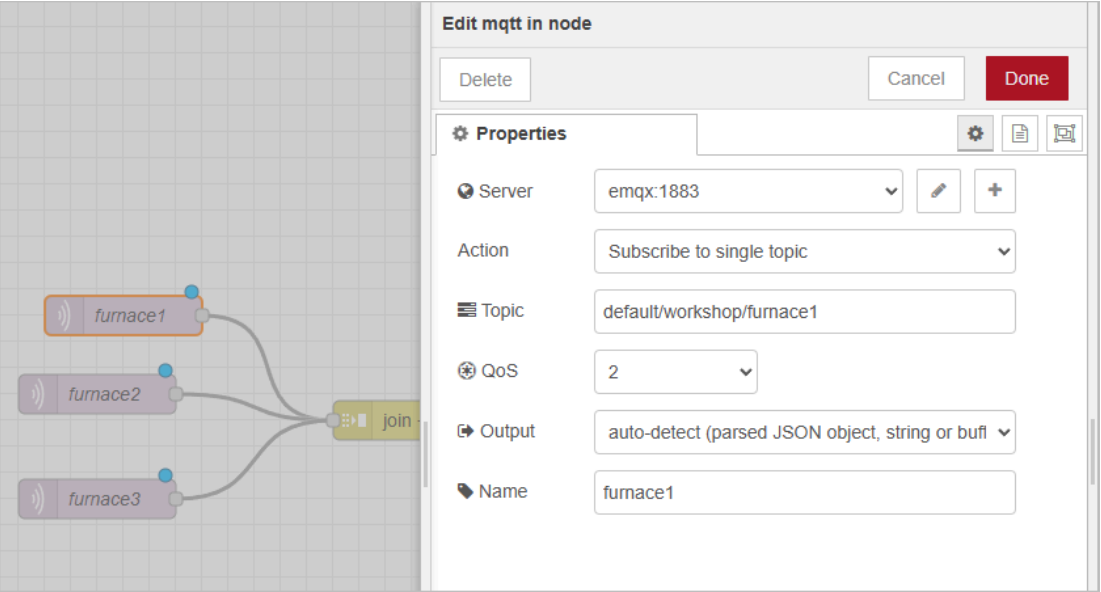
• After a message with the `msg.complete` property set

参数 (Parameter)	说明 (Description)
模式 (Mode)	合并模式。 • 手动 (manual)：自行配置消息的合并方式。 • 自动 (auto)：基于 <code>msg.parts</code> 自动重组由 拆分 (split) 节点拆分的消息序列。 • 归约序列 (reduce sequence)：对消息序列执行 JavaScript 归约函数，生成单个结果。
合并对象 (Combine each)	想要合并的消息部分。 常用为 <code>msg.payload</code>，也可选择其他（如 <code>msg.data.value</code>）。
生成结果类型 (to create)	合并后的结果类型： • 合并对象 (a merged Object)：将所有对象合并为单个对象（后接收的同名字段会覆盖先接收的）。 • 键值对象 (a key/value Object)：以 键属性 (key property)（默认 <code>msg.topic</code>）为键，合并对象 (Combine each) 对应的内容为值。 • 数组 (an Array)：按接收顺序将每个值追加到数组中。 • 字符串/二进制缓存 (a String / a Buffer)：拼接文本或二进制数据（字符串可设置分隔符；二进制缓存直接以二进制形式拼接）。
使用已有的 <code>msg.parts</code> 属性 (Use existing <code>msg.parts</code> property)	勾选后，节点会使用前置节点传入的 <code>msg.parts</code> 属性（通常由 拆分 (split) 节点生成）来确定消息分组规则，以及何时输出合并后的结果。 会用到 <code>msg.parts.count</code>、<code>msg.parts.index</code> 等字段。
发送消息 → 接收指定数量的消息片段后 (After a number of message parts)	接收到设定数量的消息片段后，输出合并后的结果。
发送消息 → 以及后续每条新消息到达时 (and every subsequent message)	勾选后，首次输出合并结果后，每有新消息到达，节点都会输出更新后的合并结果。
发送消息 → 第一条消息到达后超时触发 (After a timeout following the first message)	可选配置。第一条消息到达时启动计时器，超时（单位：秒）后，节点输出当前已收集到的所有内容的合并结果。

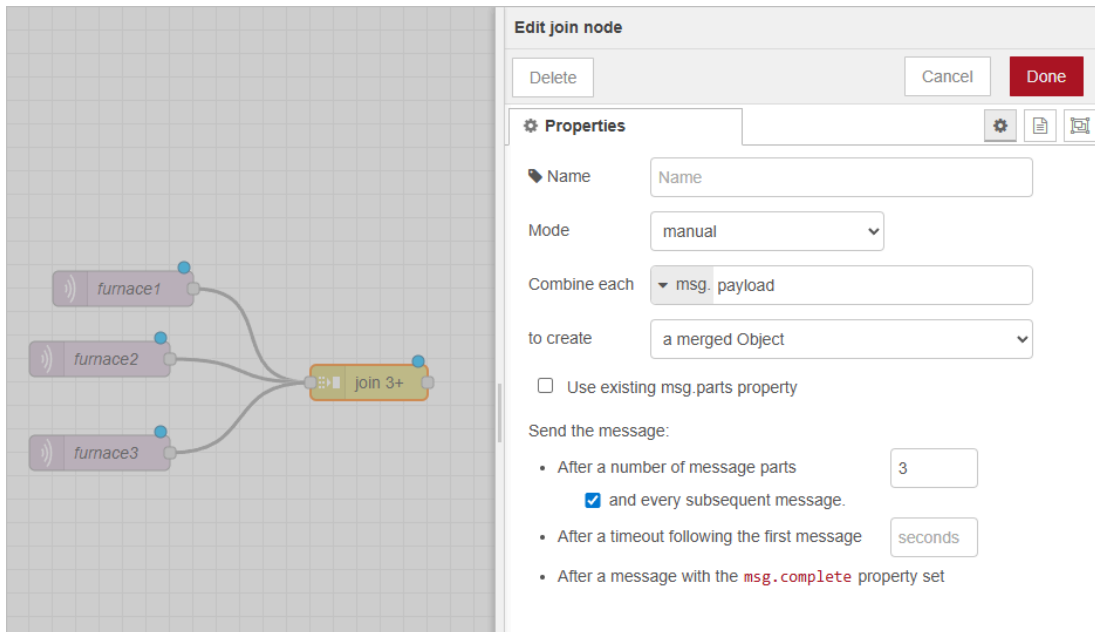
参数 (Parameter)	说明 (Description)
发送消息 → 收到含 <code>msg.complete</code> 属性的消息时 (After a message with the <code>msg.complete</code> property set)	当收到携带 <code>msg.complete = true</code> 的消息时，立即输出合并后的结果。
(高级) 字符串/二进制缓存分隔符 (String/Buffer separator)	若结果类型为 字符串 (String)，可设置可选的分隔符；若为 二进制缓存 (Buffer)，则直接拼接数据，无分隔符。

如何合并多数据源？

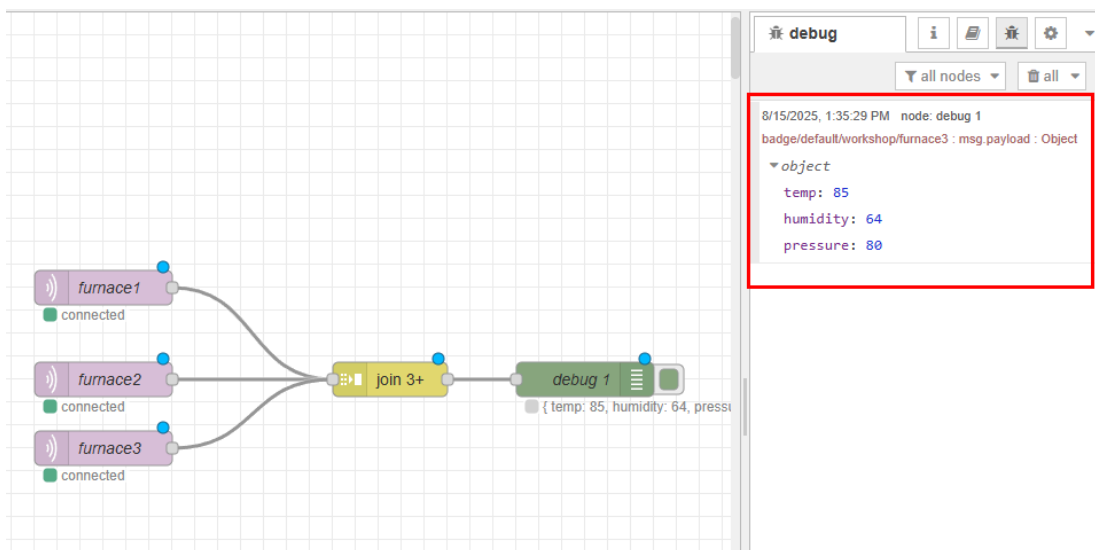
1. 在 事件流程 中拖拽 3 个 `mqtt in` 节点，分别订阅对应的主题。



2. 将 3 个节点都连接到同一个 `join` 节点，并配置为合并生成一个对象。



3. 连接 `debug` 节点，触发流程后查看合并结果。



数据过滤

示例

熔炉（furnace）数据源包含 3 个字段，我们需要根据温度阈值对消息进行路由：

- 若 温度（temp）> 85 → 路由至 输出 1（Output 1）
- 若 温度（temp）≤ 85 → 路由至 输出 2（Output 2）

```
{
  "furnace": {
    "temp": 86,
    "humidity": 64,
    "pressure": 80
  }
}
```

需使用的节点

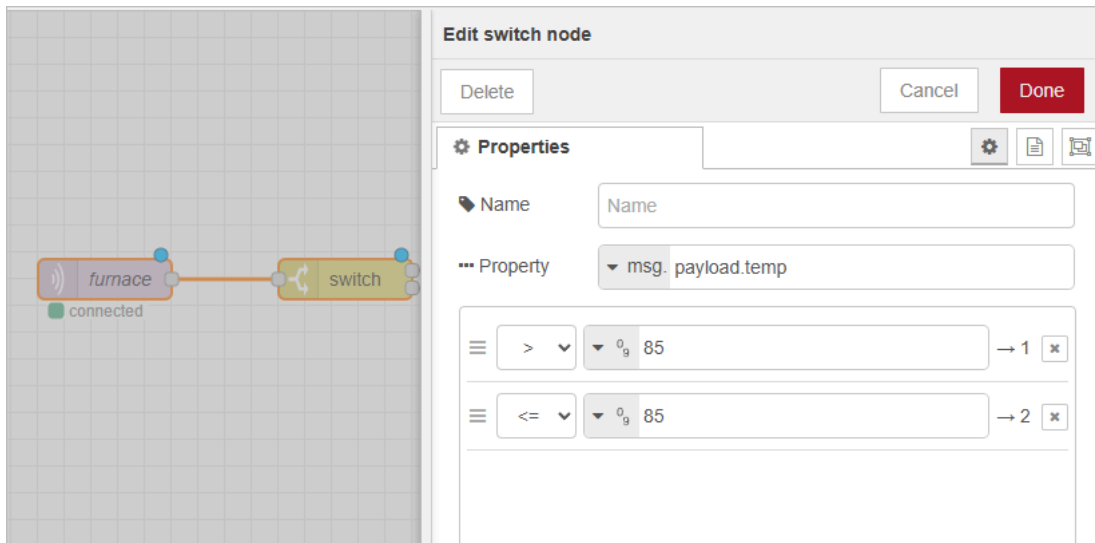
本示例中，我们使用 `switch` 节点根据条件对消息进行路由。

The screenshot shows the 'Edit switch node' configuration window. At the top, there are 'Delete', 'Cancel', and 'Done' buttons. The 'Properties' section includes a 'Name' field and a 'Property' dropdown set to 'msg.payload.temp'. Below this, a list of rules is shown. The first rule has a condition '85' and a path '→ 1'. A dropdown menu is open over the second rule, displaying 'value rules' and 'sequence rules'. The 'value rules' list includes operators like '==', '!=', '<', '<=', '>', '>=', 'has key', 'is between', 'contains', 'matches regex', 'is true', 'is false', 'is null', 'is not null', 'is of type', 'is empty', and 'is not empty'. The 'sequence rules' list includes 'head'. At the bottom, there is a checkbox labeled 'recreate message sequences'.

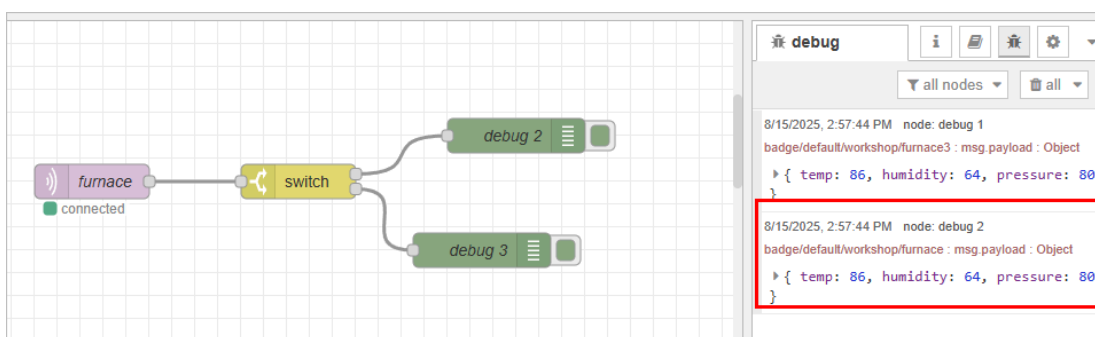
参数 (Parameter)	说明 (Description)
属性 (Property)	需评估的属性路径。常用 <code>msg.payload</code> 或更深层的属性，本示例中为 <code>msg.payload.temp</code> 。
规则 (Rules)	一组评估规则，每条规则对应一个输出端口。
类型 (Type)	比较的数据类型：数字 (number)、字符串 (string)、布尔值 (boolean)、JSONata 等。 ① 信息 确保数值比较不会被当作字符串比较处理。
值 (Value)	比较的目标值来源：常量值（如 85）、 <code>msg.</code> 字段（如 <code>msg.threshold</code> ）、 <code>flow.</code> / <code>global.</code> 上下文变量，或 <code>JSONata</code> 表达式。
检查所有规则 (checking all rules)	按顺序评估所有规则。若消息匹配多条规则，会发送至多个输出端口。
找到第一个匹配后停止 (Stop after first match)	找到第一个匹配规则后停止评估，消息仅发送至一个输出端口。
重建消息序列 (recreate message sequences)	启用后，保留并重建消息序列元数据 (<code>msg.parts</code>) —— 通常由拆分 (<code>split</code>) 节点生成—— 确保消息序列在分支后仍能保持完整。

如何筛选数据？

- 在 事件流程 中添加一个 `mqtt in` 节点，获取平台中的数据。
- 拖拽一个 `switch` 节点并配置其属性。



3. 在 **switch** 节点的每个输出端口分别连接一个 **debug** 节点。
4. 触发流程并查看结果。



更改数据

示例

我们接收到一条消息，需要重命名字段、计算新值并移除无用字段，以便后续流程更简洁地处理数据。

输入消息

```
{
  "furnace": {
    "temp_c": 30.5,
    "humidity": 64,
    "pressure": 80
  },
  "site": "A1"
}
```

目标转换要求

- 将 `furnace.temp_c` 移动至 `furnace.temperature.celsius`
- 新增 `furnace.temperature.fahrenheit` 字段（由摄氏度转换为华氏度）
- 移除 `pressure` 字段
- 将 `site` 字段值从 `A1` 修改为 `B1`

输出消息（示例）

```
{
  "furnace": {
    "humidity": 64,
    "temperature": {
      "celcius": 30.5,
      "fahrenheit": 86.9
    }
  },
  "site": "B1"
}
```

需使用的节点

此场景下，我们使用 `change` 节点对消息属性执行 设置（Set）、替换（Change）、删除（Delete）和 移动（Move）操作。

Edit change node

Delete

Cancel

Done

Properties

Name

Name

Rules

Set

▼ msg. payload

to the value

▼ a_z

×

Change

▼ msg. payload

Search for

▼ a_z

×

Replace with

▼ a_z

Delete

▼ msg. payload

×

Move

▼ msg. payload

to

▼ msg.

×

+

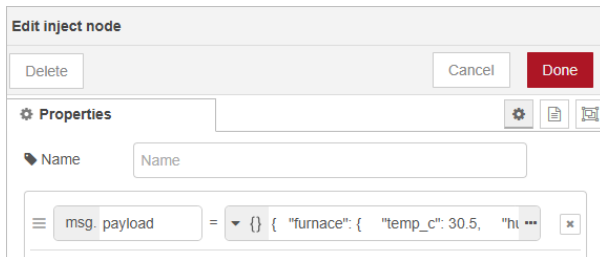
add

参数	说明
规则（Rules）	每条规则按从上到下的顺序执行。 可添加多条规则完成复杂的数据转换。
规则类型（Rule Type）	- 设置（Set）：设置或新建属性值。 - 替换（Change）：查找并替换值。 - 删除（Delete）：移除指定属性。 - 移动（Move）：将属性从一个路径移动至另一个路径。
属性/源/目标 （Property/Source/Target）	定义要操作的属性路径，例如 <code>msg.payload</code> 或 <code>msg.payload.furnace.temp_c</code> 。 也可引用 <code>flow.</code> 或 <code>global.</code> 上下文变量。
目标值（Target Value）	- 设置（Set）：要赋值的目标值。 - 替换（Change）：要查找并替换的目标值。

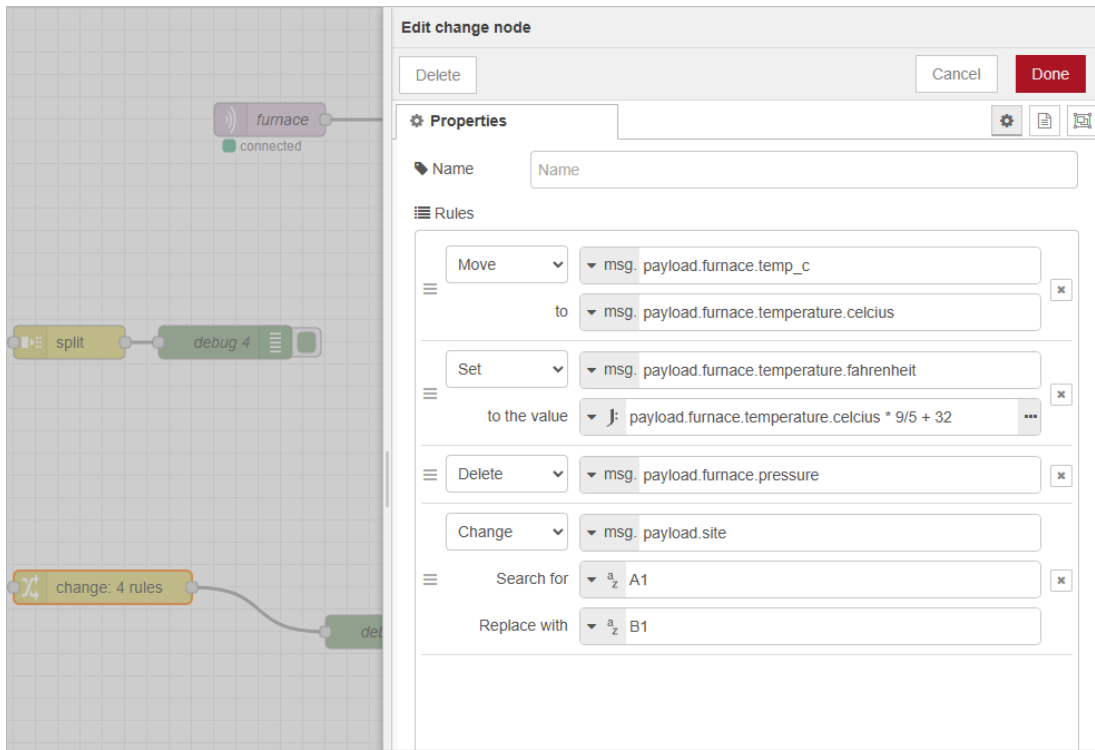
如何使用修改节点？

1. 在 事件流程 中拖拽一个 `inject` 节点，添加以下JSON作为输入内容。

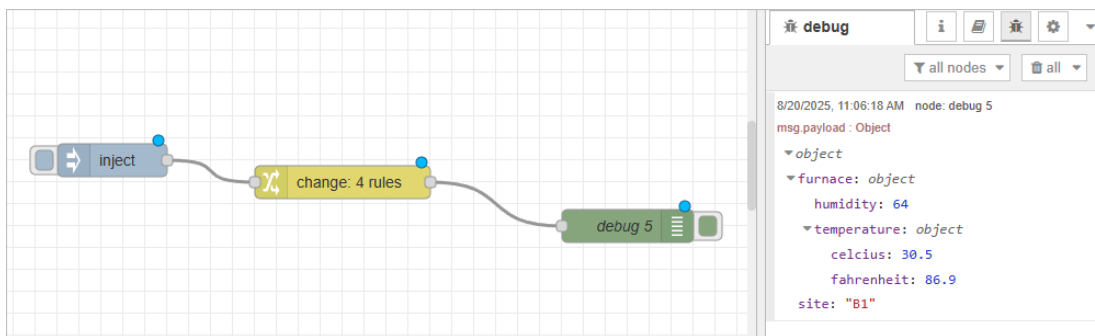
```
{
  "furnace": {
    "temp_c": 30.5,
    "humidity": 64,
    "pressure": 80
  },
  "site": "A1"
}
```



2. 将其与 `change` 节点连接。
3. 打开 `change` 节点配置面板，添加以下规则：
 - Move: `msg.payload.furnace.temp_c` → `msg.payload.furnace.temperature.celcius`
 - Set: `msg.payload.furnace.temperature.fahrenheit` → `payload.furnace.temperature.celcius * 9/5 + 32`
 - Delete: `msg.payload.pressure`
 - Change: 将 `msg.payload.site` 的值从 A1 改为 B1



4. 将 `change` 节点与 `debug` 节点连接，查看转换后的输出结果。



拆分数据

示例

我们有一条包含多组熔炉读数的消息，需要将其拆分为多条独立的消息，以便进行后续处理。

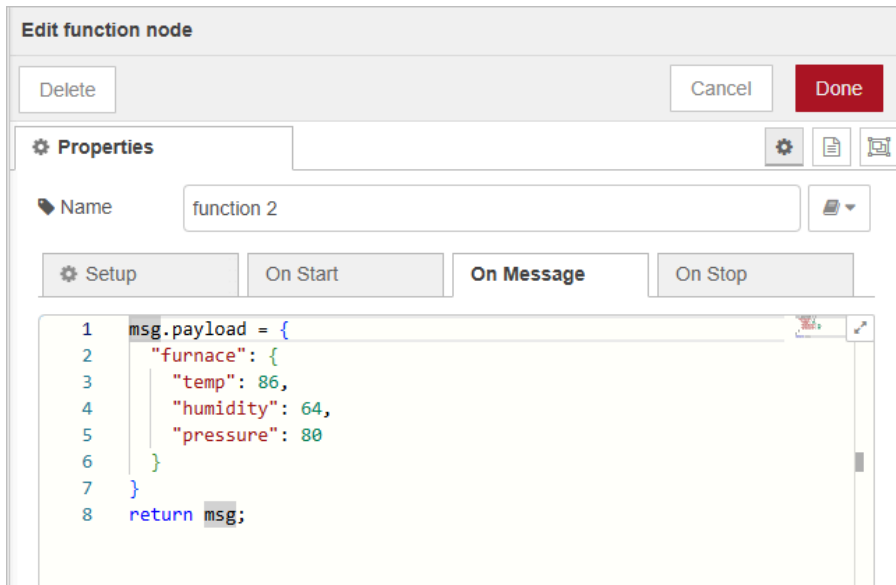
```

{
  "furnace": {
    "temp": 86,
    "humidity": 64,
    "pressure": 80
  }
}
  
```

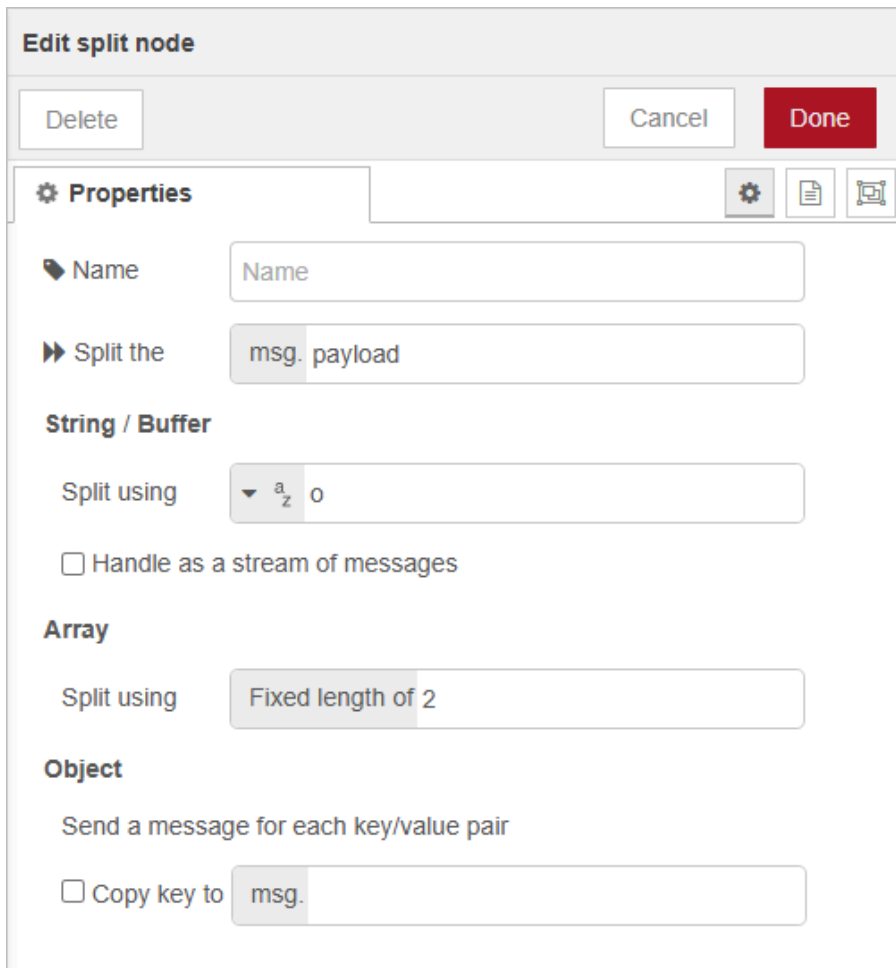

参数 (Parameter)	说明 (Description)
名称 (Name)	节点的可选标签，用于在流程中标识该节点。
拆分对象 (Split the)	要拆分的消息属性，通常为 <code>msg.payload</code> ，也可以是其他属性（如 <code>msg.payload.data</code> ）。
字符串/二进制缓存 - 拆分方式 (String / Buffer - Split using)	针对字符串或二进制缓存类型的负载，指定拆分方式： • 字符串 (string)：使用特定字符串或正则表达式模式拆分消息。 • 二进制缓存 (buffer)：按设定的块大小拆分消息。 • 固定长度 (Fixed length of)：按设定的长度拆分字符串。 • 留空则不拆分字符串。
作为消息流处理 (Handle as a stream of messages)	勾选后，节点会将传入数据视为连续流，数据到达时立即发送拆分后的消息，而非等待完整负载接收完毕。
数组 - 拆分方式 (Array - Split using)	固定长度 N (Fixed length of N)：将数组拆分为多条消息，每条消息包含 N 个元素。
对象 - 为每个键值对发送一条消息 (Object - Send a message for each key/value pair)	每个键值对的值将作为拆分后消息的负载 (payload) 返回。
将键复制到 <code>msg.payload</code> (Copy key to <code>msg.payload</code>)	勾选后，原始键名会作为拆分后消息的 <code>msg.payload</code> 返回。

如何拆分消息？

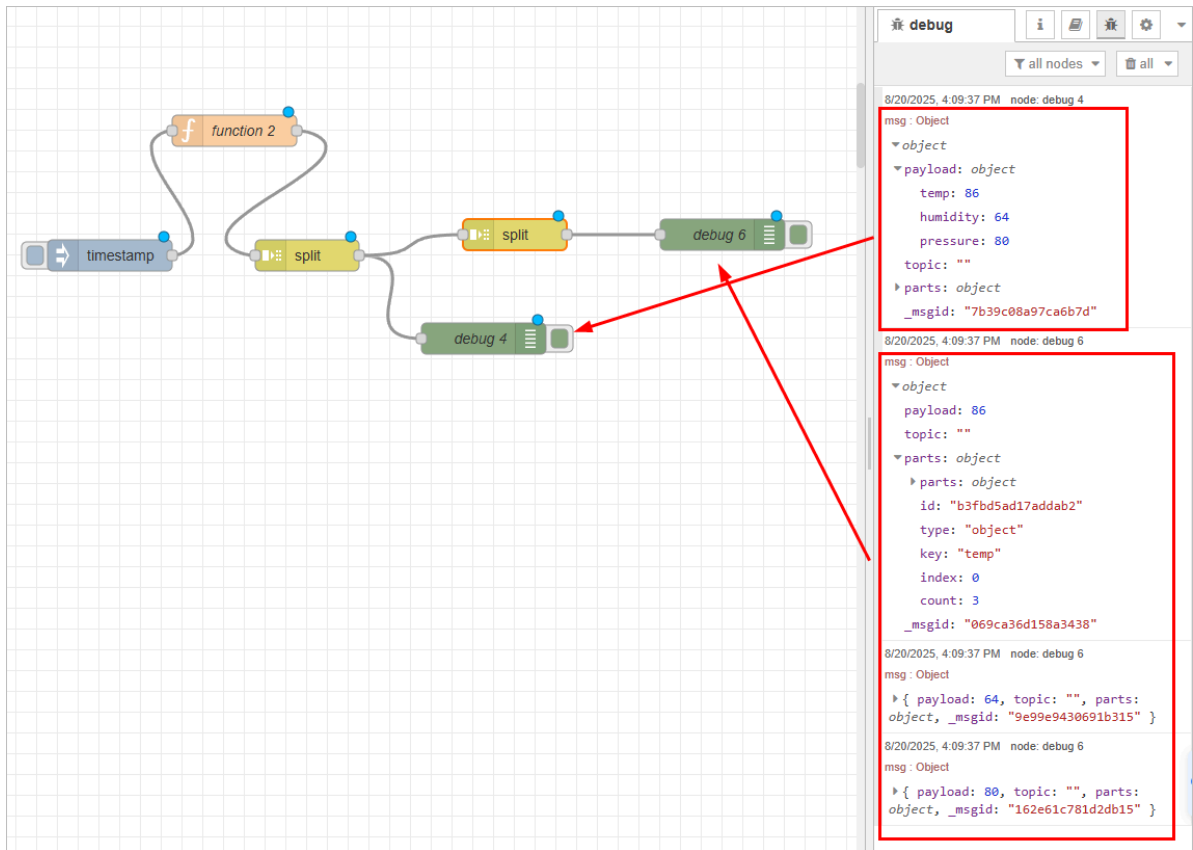
1. 在 事件流程 中拖拽一个 `inject` 节点，再通过一个 `function` 节点传递目标对象数据。



- 按顺序将其连接到 2 个 `split` 节点，并为两个节点配置一样的「对象 (Object)」属性。



- 将 `split` 节点与 `debug` 节点连接，查看每条拆分后的输出消息。

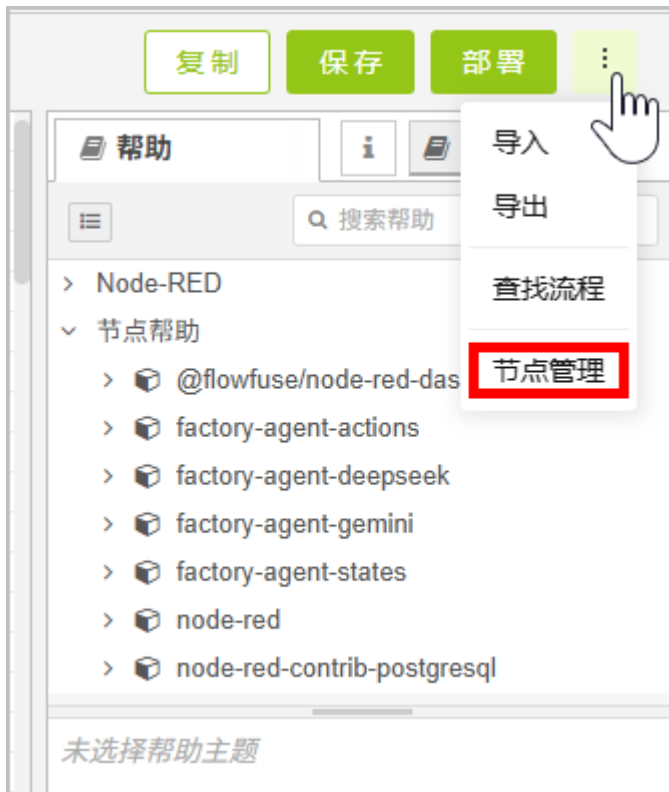


数据可视化

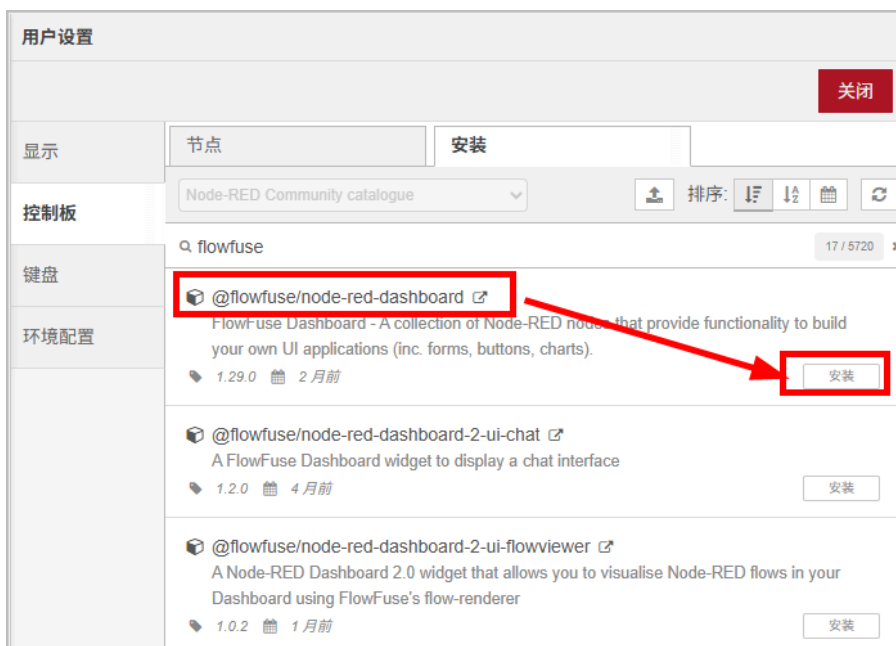
如何用事件流程构建数据看板

采集至平台并经过处理的数据，可在平台中以自定义样式的图表、表格、文本等格式进行可视化展示。

1. 登录平台，选择 数据管理 > 事件流程。
2. 单击 新建事件流程 添加一条流。
3. 单击流名称开始编辑。
4. 单击右上角  图标，然后单击 节点管理。



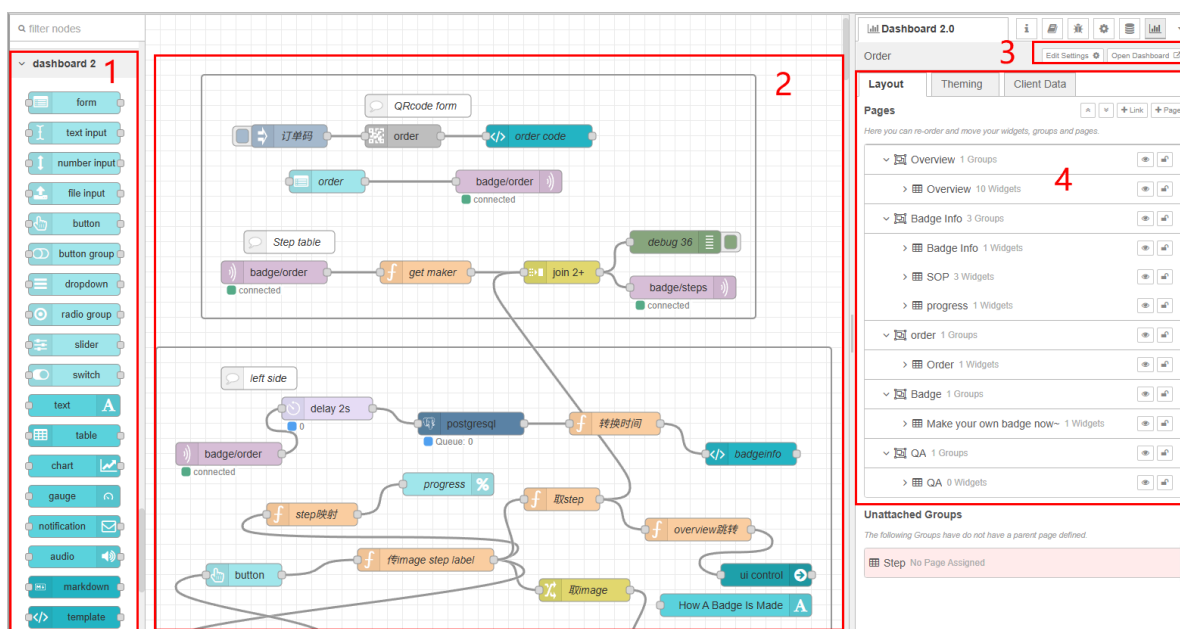
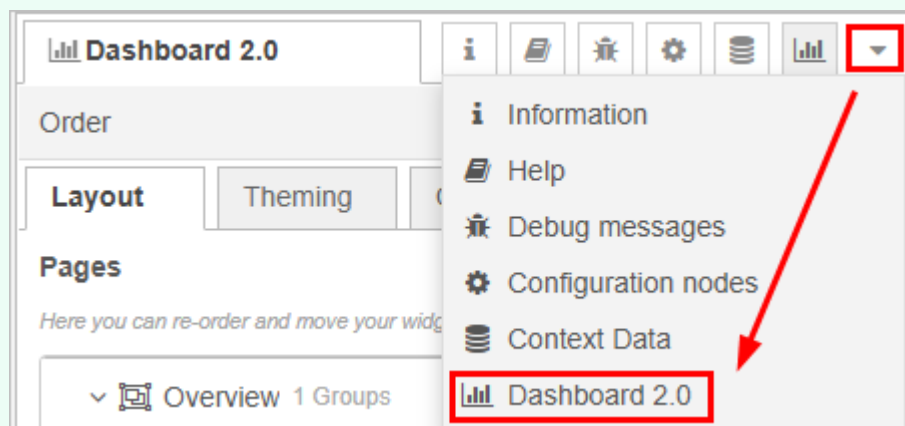
5. 在 安装 页面搜索并安装 `@flowfuse/node-red-dashboard` 节点。



6. 使用 flowfuse 的节点构建数据看板：

✓ 提示

单击右上角  图标，然后单击 Dashboard2.0 可调整看板布局。



序号	项目	说明
1	节点库	Dashboard 2.0 相关节点，包括仪表盘、表单、进度条等。
2	画布	拖拽节点构建数据看板的操作区域。
3	操作栏	编辑 <code>ui-base</code> 节点及预览数据看板。
4	布局面板	按层级布局显示所有已添加的节点。

① 信息

各节点详细使用方法请参考 [Flowfuse 节点文档](#)。

数据看板展示

如何在数据看板中展示数据

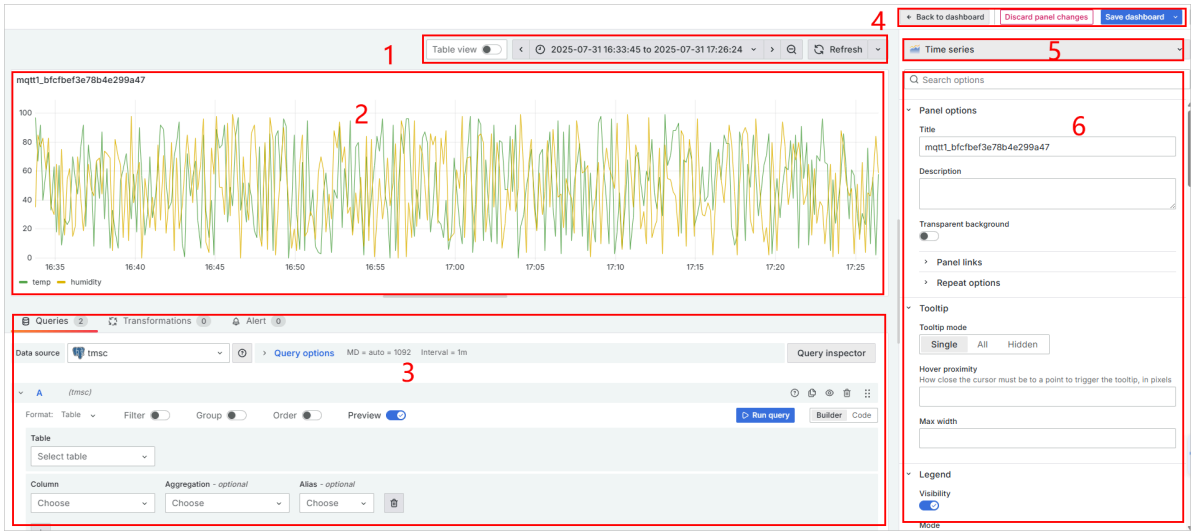
采集至平台并经过处理的数据，可在数据看板中以图表、表格、热力图等多种格式进行可视化展示，助力全面洞察数据价值。

1. 登录平台，选择 数据管理 > 数据看板。



序号	项目	说明
1	搜索	搜索已存在的数据看板。
	新建数据看板	添加一个新的数据看板。
2	数据看板列表	显示数据看板信息，单击名称可开始编辑。
3	显示方式	切换数据看板的卡片式/列表式显示。
4	操作	对数据看板进行复制/编辑/删除操作。

2. 单击 新建数据看板 添加一个数据看板。
3. 单击数据看板名称开始编辑。
4. 单击 Add visualization，在弹出窗口中为数据看板选择数据源：
- 默认的 tmsc 是存储 数据建模 中数据的内置数据库。
 - 单击右下角 Configure a new data source 可配置其他数据源（如 MQTT）。



序号	项目	说明
1	工具栏	切换图表视图与表格视图。
	时间范围	选择数据展示的时间范围。
	刷新	刷新当前可视化元素。
2	展示区域	通过图元展示当前数据源的数据。
3	数据源配置	更换数据源，可从数据库中选择列，或直接通过 SQL 查询处理复杂数据。
4	通用操作	返回数据看板列表/放弃当前面板的修改/保存数据看板。
5	元素库	选择不同的元素用于数据展示。
6	元素配置	当前选中元素的相关配置。

从 Tier0 获取数据

数据可通过多种协议采集至平台，也可通过多种方式传递给第三方。

通过MQTT订阅获取数据

① 信息

该方式适用于具备MQTT协议使用经验的第三方。

1. 登录平台，选择 数据管理 > 事件流程。
2. 单击 新建事件流程，输入名称后单击 保存。
3. 单击刚创建的事件流程，拖拽一个 `mqtt in` 节点并配置参数：
 - 平台MQTT代理地址： `emqx:1883`
 - 主题：需获取数据的文件路径（例如 `factory/workshop/order`）。

4. 对获取到的数据进行处理，或直接使用 `mqtt out` 节点将数据传输至第三方MQTT。

5. 部署流程，当订阅主题数据发生变化时，数据将自动推送至第三方。

通过API获取数据

① 信息

该方式适用于不支持MQTT协议、需要事件驱动型数据的第三方。

1. 登录平台，选择 数据管理 > 事件流程。
2. 单击 新建事件流程，输入名称后单击 保存。
3. 单击刚创建的事件流程，拖拽一个 `mqtt in` 节点并配置参数：
 - 平台MQTT代理地址： `emqx:1883`
 - 主题：需获取数据的文件路径（例如 `factory/workshop/order`）。

The screenshot shows the 'Edit mqtt in node' configuration window. It includes a 'Delete' button, 'Cancel', and 'Done' buttons. The 'Properties' section contains the following fields:

- Server:** A dropdown menu set to 'emqx'.
- Action:** A dropdown menu set to 'Subscribe to single topic'.
- Topic:** A text input field containing 'factory/workshop/order'.
- QoS:** A dropdown menu set to '2'.
- Output:** A dropdown menu set to 'auto-detect (parsed JSON object, string or buffer)'.
- Name:** A text input field containing 'Name'.

4. 拖拽一个 `function` 节点至画布，编写脚本将获取到的数据按第三方接收API的要求格式化。

```
// 若负载为字符串类型，则解析为JSON
if (typeof msg.payload === "string") {
  msg.payload = JSON.parse(msg.payload);
}

// 设置请求头
msg.headers = {
  "Content-Type": "application/json"
};

return msg;
```

5. 使用 `HTTP request` 节点通过API将数据发送至第三方。

Dialog box titled "Edit http request node" with buttons "Delete", "Cancel", and "Done".

Properties

- Method:** POST
- URL:** http://thirdparty.api.url/data/destination

Options (all unchecked):

- Enable secure (SSL/TLS) connection
- Use authentication
- Enable connection keep-alive
- Use proxy
- Only send non-2xx responses to Catch node
- Disable strict HTTP parsing

Return: a UTF-8 string

Headers:

+ add

Name: Name

6. 部署流程，当订阅主题数据发生变化时，数据将自动推送至第三方。

直接从平台数据库获取数据

① 信息

该方式适用于需要查询历史数据的场景。

1. 登录平台，选择 数据管理 > 事件流程。
2. 单击 新建事件流程，输入名称后单击 保存。
3. 单击刚创建的事件流程，拖拽一个 `inject` 节点。
4. 拖拽一个 `PostgreSQL` 节点至画布，连接平台PostgreSQL数据库：
 - Connection
 - 主机: postgresql
 - 端口: 5432
 - 数据库: postgres

- SSL: false
- Security
 - 用户名: postgres
 - 密码: postgres

The screenshot shows the 'Edit postgresql node' configuration window with the 'Connection' tab selected. The 'Name' field is 'dbConnection'. The 'Connection' tab contains the following fields: Host (postgresql), Port (5432), Database (postgres), and SSL (false). These fields are highlighted with a red rectangle.

The screenshot shows the 'Edit postgresql node' configuration window with the 'Security' tab selected. The 'Name' field is 'dbConnection'. The 'Security' tab contains the following fields: User (postgres) and Password (masked with dots). These fields are highlighted with a red rectangle.

✓ 提示

若未安装 PostgreSQL 节点，可在节点管理中搜索 `node-red-contrib-postgresql` 进行安装。

5. 编写SQL语句查询平台数据库中的数据：

① 信息

表名可从 数据建模 下对应文件的明细中获取。

CNC	
Topic	Equipment/状态/CNC
别名	_CNC_4fd9a75ff6ac4a5f9069
显示名称	
描述	
引用模板	
数据类型	关系型
标签	

SELECT * FROM 表名;

Edit postgresql node

Delete Cancel Done

Properties

Name: Name

Server: pg

☐ Split results in multiple messages

Number of rows per message: 1

Query

1 SELECT * FROM _order_0baa755c15544f81a302;

6. 添加 HTTP request 或 mqtt out 节点，将查询到的数据传输至第三方。

高阶使用

容器管理

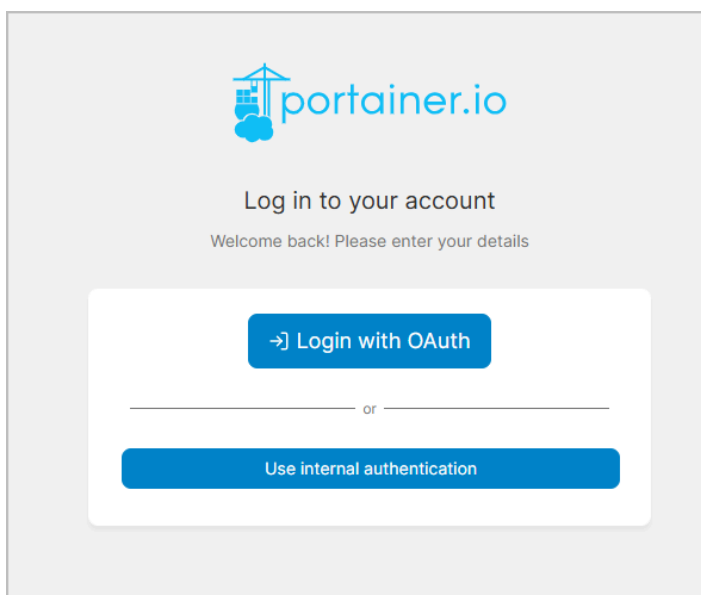
Tier0 采用 Portainer 进行容器管理，可轻松实现应用的部署、管理与扩缩容。

① 信息

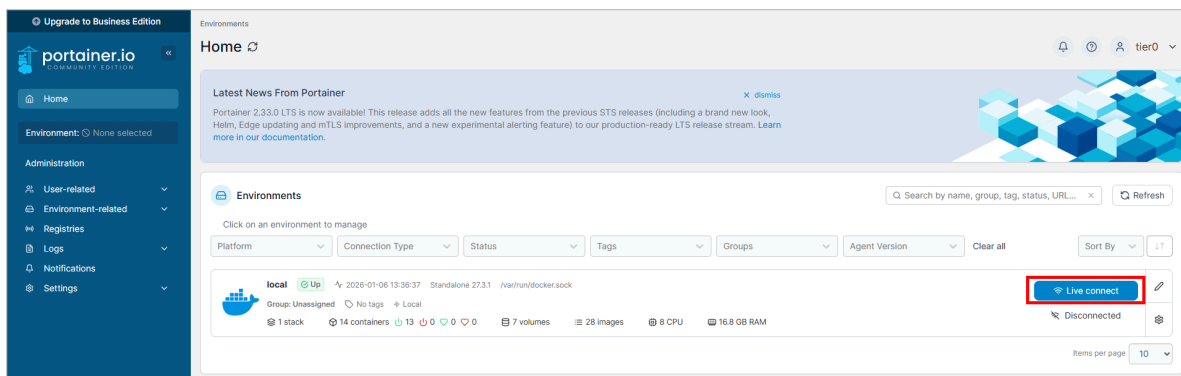
本指引仅介绍通用容器管理的简单操作。如需了解 Portainer 的详细用法，请参阅 [Portainer 官方文档](#)。

进入容器管理页面

1. 登录 Tier0 平台，然后依次选择 工具集 > 容器管理。

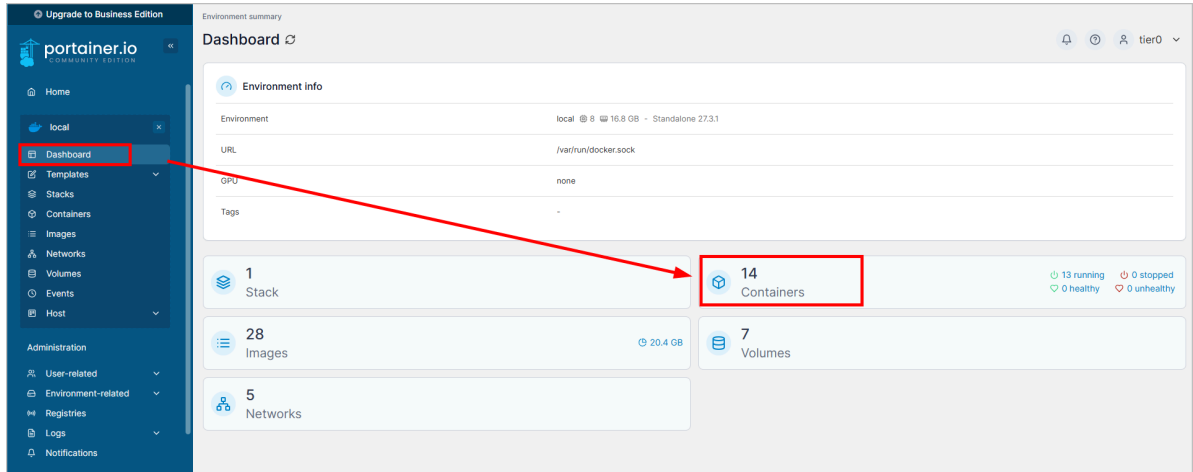


2. 单击 Login with OAuth，再单击 Live connect，即可开始管理容器。

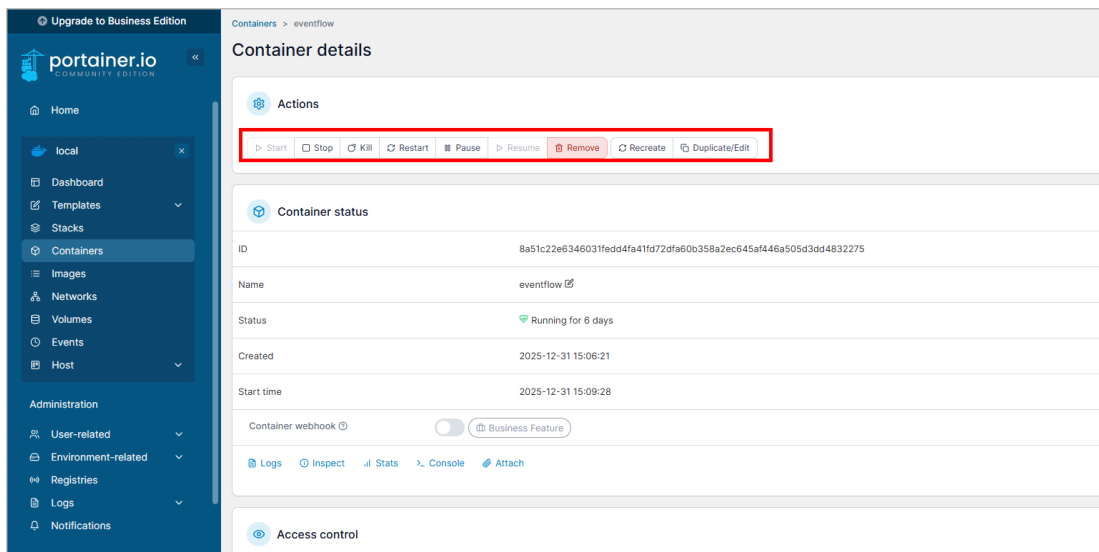


容器通用操作

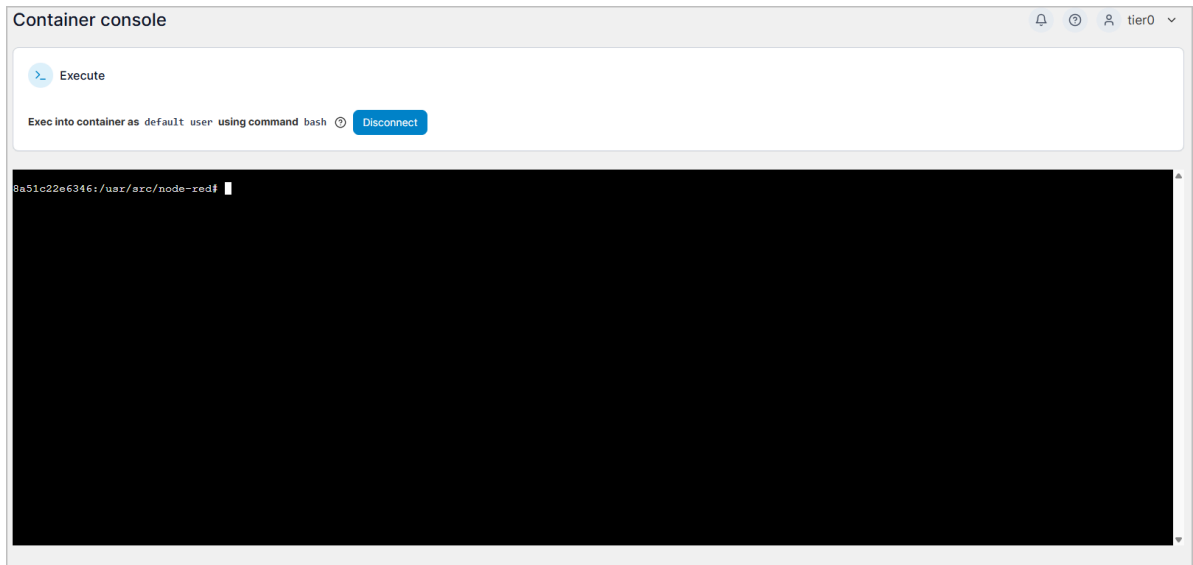
1. 在 Dashboard 页面，单击 Containers。



2. 选中某个容器后，即可对其执行启动、停止、重启、暂停或删除操作。

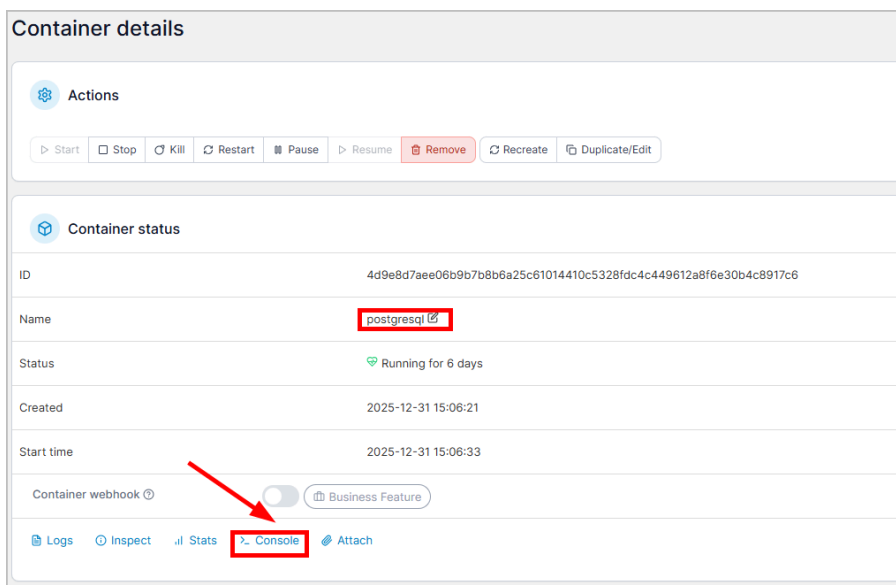


3. （可选操作）单击 Console，进入容器终端，执行更复杂的操作。



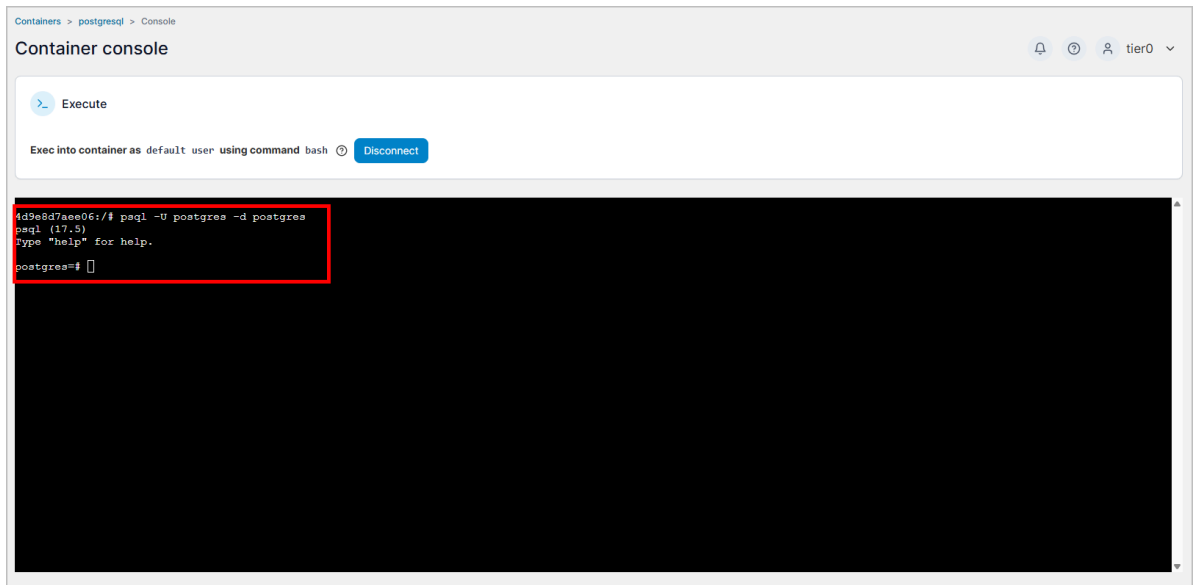
访问容器中的数据库

1. 在 Dashboard 页面，单击 Containers。
2. 切换至第 2 页，选中 postgresql，然后单击 Console。



3. 单击 Connect，然后执行以下命令访问数据库。

```
psql -U postgres -d postgres
```



Notebook

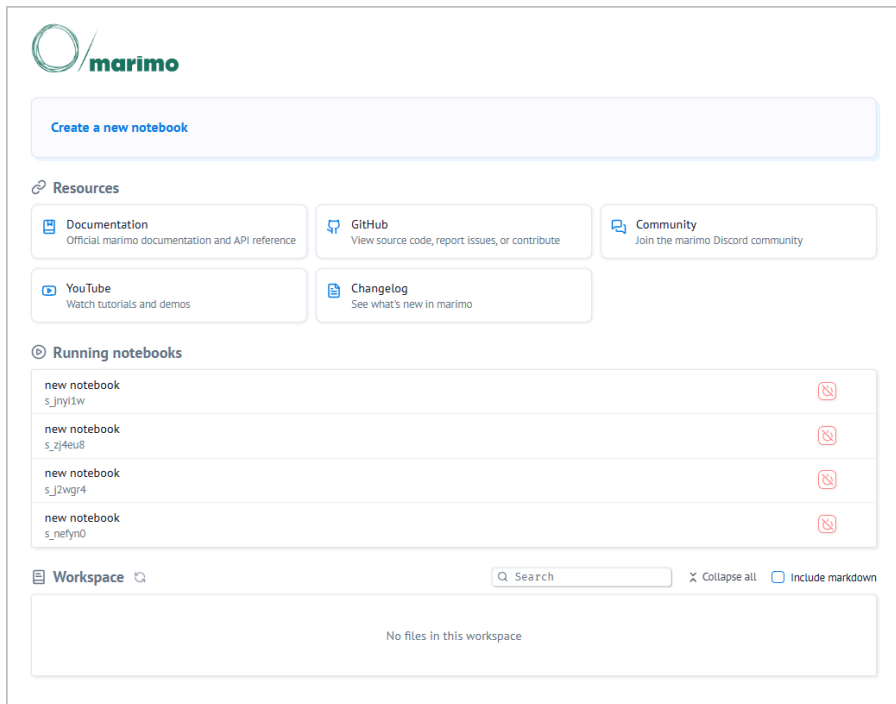
平台集成了 Marimo 工具，可便捷地使用 Python 脚本。例如，你可基于数据建模 中已连接的数据，完成模型训练、SQL 查询执行、仪表盘设计等操作。

借助 Notebook 运行 SQL 查询

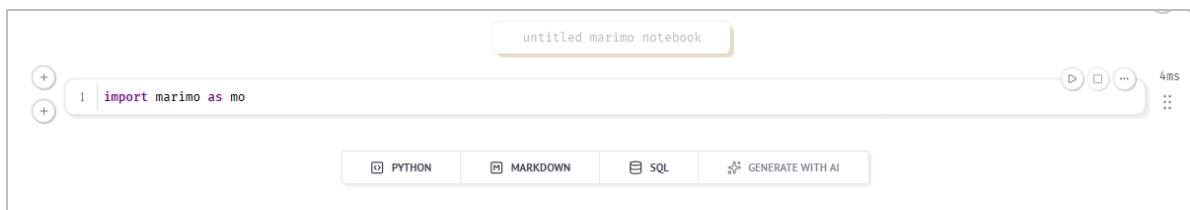
① 备注

平台数据库已连接至Notebook，可直接访问使用。

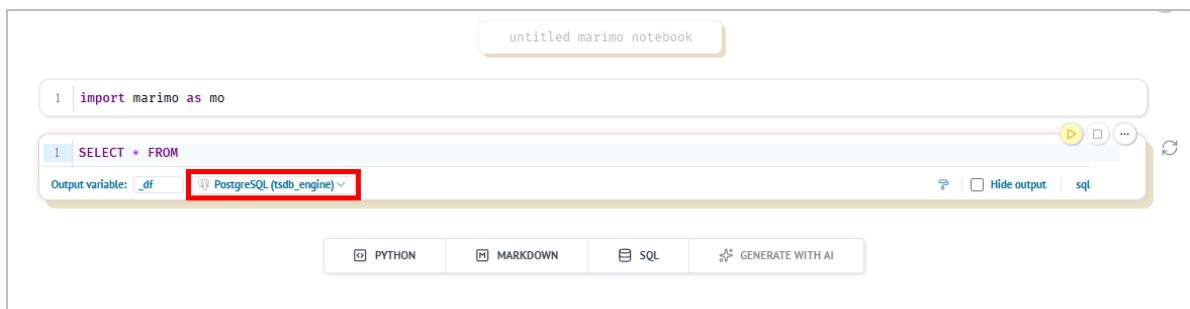
1. 登录平台后，选择 工具集 > Notebook。



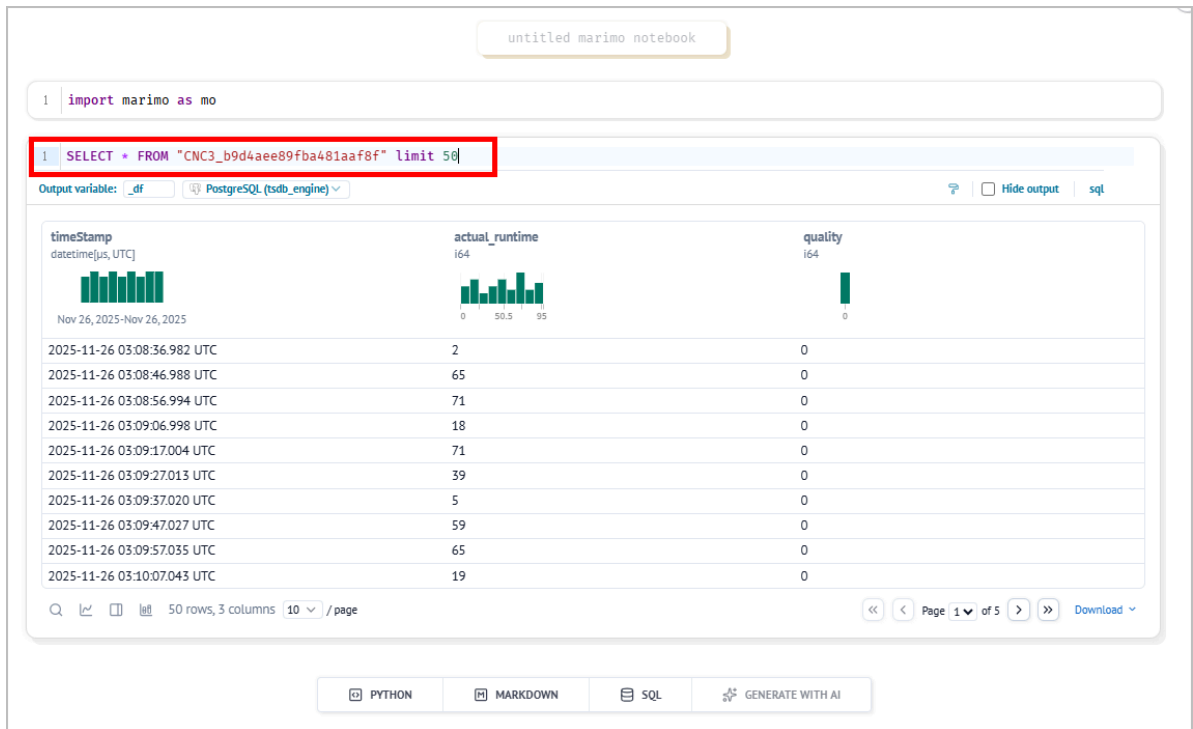
2. 单击 `Create a new notebook`，再单击  运行首个代码单元格。



3. 单击 `SQL` 新增一个SQL单元格，选择已连接的数据库后执行 `SQL` 语句。



4. 从数据建模 下的文件 明细 中获取数据表名称（即文件别名），开始编写 `SQL` 语句。



✓ 提示

- 单击左侧面板的 ，可查看已连接的数据库列表。
- 找到所需的数据表后，单击表名旁的 ，可自动生成 SQL 语句。

借助 Notebook 绘制图表

1. 登录平台后，选择 工具集 > Notebook。
2. 单击 Create a new notebook，新增以下 3 个单元格，绘制简易折线图。

- 单元格 1：导入相关库

```
import marimo as mo
```

- 单元格 2：从数据库中查询数据

```
_df = mo.sql(  
    f"""  
    SELECT *  
    FROM "CNC3_b9d4aee89fba481aaf8f"  
    LIMIT 100  
    """,  
    engine=tsdb_engine  
)  
data = _df
```

✓ 提示

使用 SQL 单元格编写查询语句，然后单击单元格末尾的 ，将该单元格切换为 Python 模式。务必将查询结果赋值给一个变量（例如 `data`）。

- 单元格 3：绘制折线图

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.plot(data["actual_runtime"])
plt.title("Actual Runtime")
plt.xlabel("Index")
plt.ylabel("actual_runtime")
plt.grid(True)
plt.show()
```

3. 单击右下角的 ，运行所有代码单元格并查看生成的折线图。

```
1 import marimo as mo
```

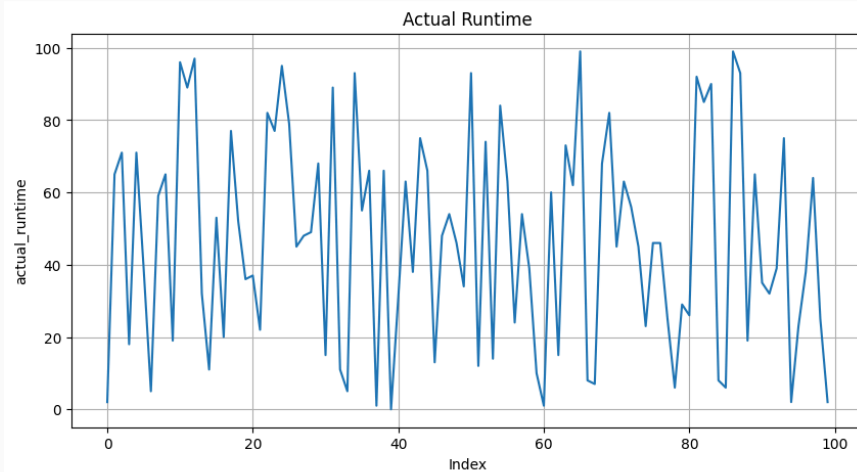
```
1 v _df = mo.sql(  
2 v     f"""  
3     SELECT *  
4     FROM "CNC3_b9d4aee89fba481aaf8f"  
5     LIMIT 100  
6     """  
7     ,  
8     engine=tsdb_engine  
9 )  
data = _df
```

timeStamp	actual_runtime	quality
datetime[us,UTC]	i64	i64
Nov 26, 2025-Nov 26, 2025	0 46 93	0
2025-11-26 03:08:36.982 UTC	2	0
2025-11-26 03:08:46.988 UTC	65	0
2025-11-26 03:08:56.994 UTC	71	0
2025-11-26 03:09:06.998 UTC	18	0
2025-11-26 03:09:17.004 UTC	71	0
2025-11-26 03:09:27.013 UTC	39	0
2025-11-26 03:09:37.020 UTC	5	0
2025-11-26 03:09:47.027 UTC	59	0
2025-11-26 03:09:57.035 UTC	65	0
2025-11-26 03:10:07.043 UTC	19	0

100 rows, 3 columns 10 / page

Page 1 of 10 Download

```
1 import matplotlib.pyplot as plt  
2  
3 plt.figure(figsize=(10,5))  
4 plt.plot(data["actual_runtime"])  
5 plt.title("Actual Runtime")  
6 plt.xlabel("Index")  
7 plt.ylabel("actual_runtime")  
8 plt.grid(True)  
9 plt.show()  
10  
11
```



PYTHON

MARKDOWN

SQL

GENERATE WITH AI

高阶使用

Tier0 支持使用与其深度集成的开源产品，并赋予全部操作权限。

1. 登录 Tier0 平台，选择工具集 > 高阶使用。



2. 单击任一开源产品，即可进入其管理页面。



系统管理

路由管理

服务路由配置

平台基于 Konga 管理服务路由，你可将自有应用接入平台实现集成。遵循相关规则，你可完成以下操作：

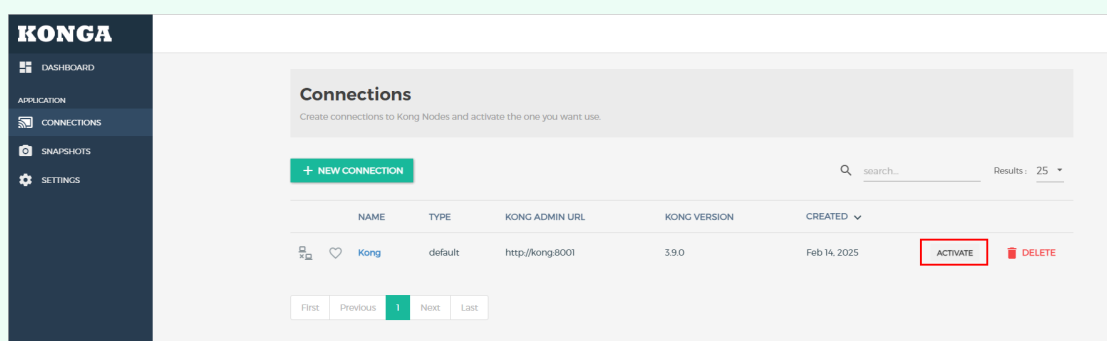
- 配置后端服务的路由路径
- 配置访问授权机制
- 将外部服务作为应用接入平台前端

创建服务

1. 选择 系统管理 > 路由管理，单击 SERVICES。

✓ 提示

若 DASHBOARD 显示无活跃连接，请进入 CONNECTIONS，单击 Kong 旁的 ACTIVATE。



2. 单击 ADD NEW SERVICE，输入后端服务信息。

① 信息

以下为必填参数。例如，你的后端资源为 <https://example.com:8080>

- Protocol: https
- Host: example.com
- Port: 8080
- Path: 请求路径前缀，会在客户端请求转发至服务前添加到请求路径中

CREATE SERVICE

Name

(optional)

The service name.

Description

(optional)

An optional service description.

Tags

(optional)

Optionally add tags to the service

Url

(shorthand-attribute)

Shorthand attribute to set `protocol`, `host`, `port` and `path` at once. This attribute is write-only (the Admin API never "returns" the url).

Protocol

(semi-optional)

The protocol used to communicate with the upstream. It can be one of `http` or `https`.

Host

(semi-optional)

The host of the upstream server.

Port

(semi-optional)

The upstream server port. Defaults to `80`.

Path

(optional)

The path to be used in requests to the upstream server. Empty by default.

Retries

(optional)

5

The number of retries to execute upon failure to proxy. The default is `5`.

3. 单击 SUBMIT SERVICE。

建立路由

为服务配置访问路由路径：

1. 在 Service Details 页面，单击 Routes。
2. 单击 ADD ROUTE，输入服务访问路由信息。

- **Tags**: 部分标签已映射固定含义。

标签	说明
menu	服务将作为应用显示在平台首页
parentName:menu.tag.xxx	xxx 为平台现有一级菜单名称（全小写），表示当前路由作为 xxx 的二级菜单显示
homeParentName:menu.tag.xxx	xxx 为平台现有一级菜单名称（全小写），表示当前路由显示在导航栏中，作为 xxx 的二级菜单
description:menu.desc.xxx	xxx 为路由名称，表示 xxx 对应的描述信息

- **Paths**: Kong 可识别的请求路径，Kong 会匹配该路径并将请求转发至你的服务。

ADD ROUTE TO MY-BACKEND

✕

* For hosts, paths, methods and protocols, snis, sources, headers and destinations press enter to apply every value you type

Name

(optional)

The name of the Route.

Tags

(optional)

Optionally add tags to the route

Hosts

(semi-optional)

A list of domain names that match this Route. For example: example.com. At least one of hosts, paths, or methods must be set.

Paths

(semi-optional)

A list of paths that match this Route. For example: /my-path. At least one of `hosts`, `paths`, or `methods` must be set.

Headers

(semi-optional)

One or more lists of values indexed by header name that will cause this Route to match if present in the request. The `Host` header cannot be used with this attribute: hosts should be specified using the `hosts` attribute.
 Field values format example: `x-some-header:foo,bar`

Path handling

v1

▼

Controls how the Service path, Route path and requested path are combined when sending a request to the upstream. See above for a detailed description of each behavior. Accepted values are: `"v0"`, `"v1"`. Defaults to `"v1"`.

Https redirect status code

(optional)

426

The status code Kong responds with when all properties of a Route match except the protocol, i.e. if the protocol of the request is `HTTP` instead of `HTTPS`. `Location` header is injected by Kong if the field is set to 301, 302, 307 or 308. Defaults to `426`.

3. 单击 SUBMIT ROUTE。

配置认证机制

为服务配置认证机制：

1. 在 Service Details 页面，单击 Plugins。
2. 单击 ADD PLUGIN，选择 Key Auth。

① 备注

支持添加其他认证方式，详情请参考 [Konga 官方文档](#)。

3. 输入 Key names，指定认证信息的传输方式。

Add Key Authentication (also referred to as an API key) to your APIs. Consumers then add their key either in a querystring parameter or a header to authenticate their requests.

consumer

The CONSUMER ID that this plugin configuration will target. This value can only be used if authentication has been enabled so that the system can identify the user making the request. If left blank, the plugin will be applied to all consumers.

key names

Tip: Press **Enter** to accept a value.

Describes an array of comma separated parameter names where the plugin will look for a key. The client must send the authentication key in one of those key names, and the plugin will try to read the credential from a header or the querystring parameter with the same name.

hide credentials

☐ NO

An optional boolean value telling the plugin to hide the credential to the upstream API server. It will be removed by Kong before proxying the request.

anonymous

key in header ☒ YES

key in query ☒ YES

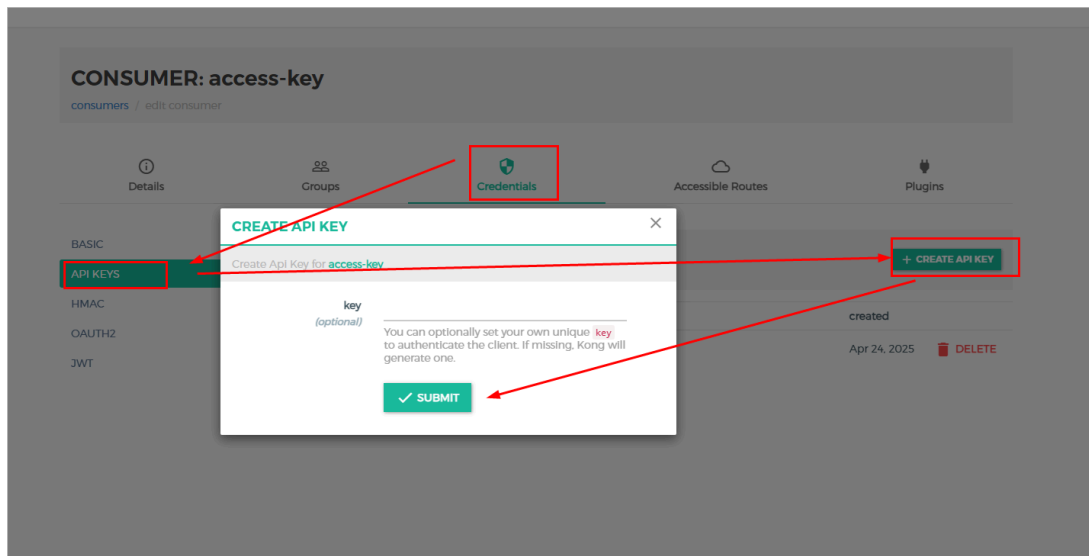
key in body ☐ NO

run on preflight ☒ YES

realm

4. 单击 ADD PLUGIN 并退出。
5. 单击左侧菜单 CONSUMERS，添加认证密钥：

1. 单击 CREATE CONSUMER，设置用户名或 ID。
2. 在详情页，选择 Credentials。
3. 选择 API KEYS，创建 API 密钥。



4. 留空并单击 SUBMIT，Konga 会自动生成自定义密钥，访问服务时需使用该密钥。

用户管理

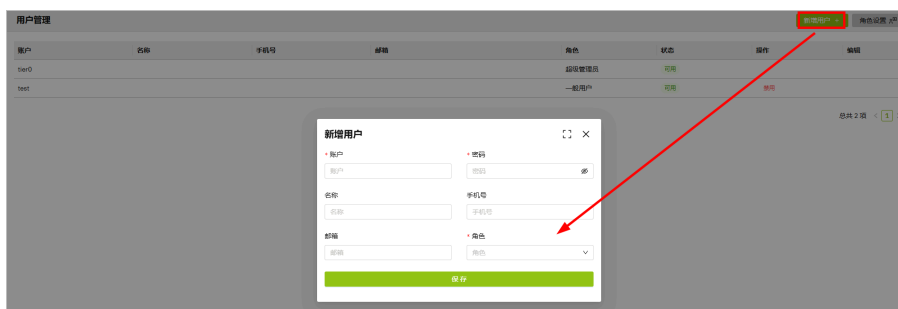
在平台中，你可以新增用户并为其分配不同角色，便捷管理模块访问权限。同时，也支持用户自行注册账号。

① 信息

平台采用 Keycloak 实现全面的用户管理功能，你可通过 **系统管理 > 身份与访问管理** 进行自定义配置。

新增用户

1. 登录平台后，选择 **系统管理 > 用户管理**。
2. 单击右上角的 **新增用户**。



3. 填写用户信息并为其选择角色，单击 保存。

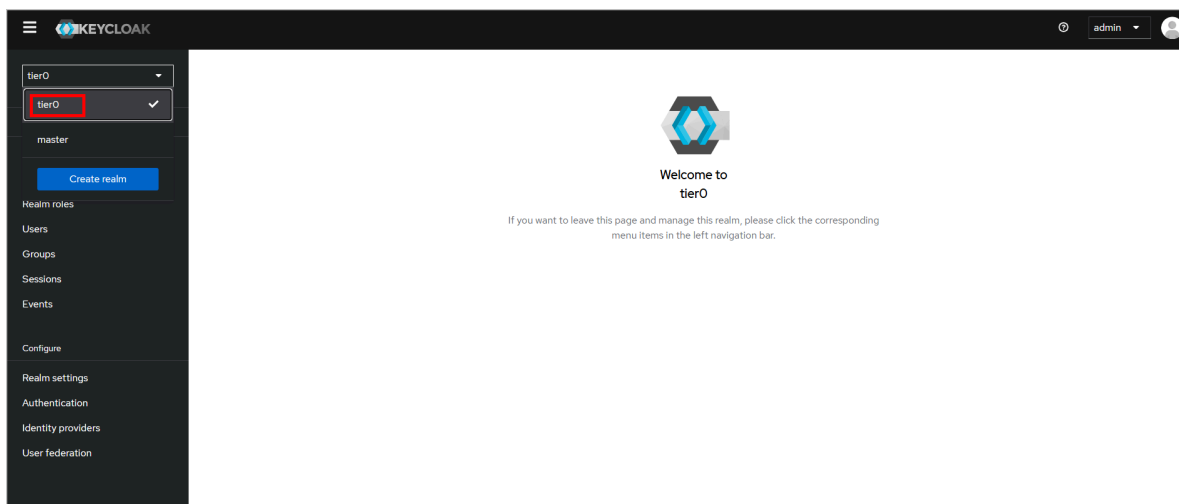
账户	名称	手机号	邮箱	角色	状态	操作	编辑
tier0				超级管理员	可用		
test				一般用户	可用	禁用	

- 支持对用户进行禁用、编辑、删除及密码重置操作。
- Keycloak 中预设的默认用户无法禁用或删除。

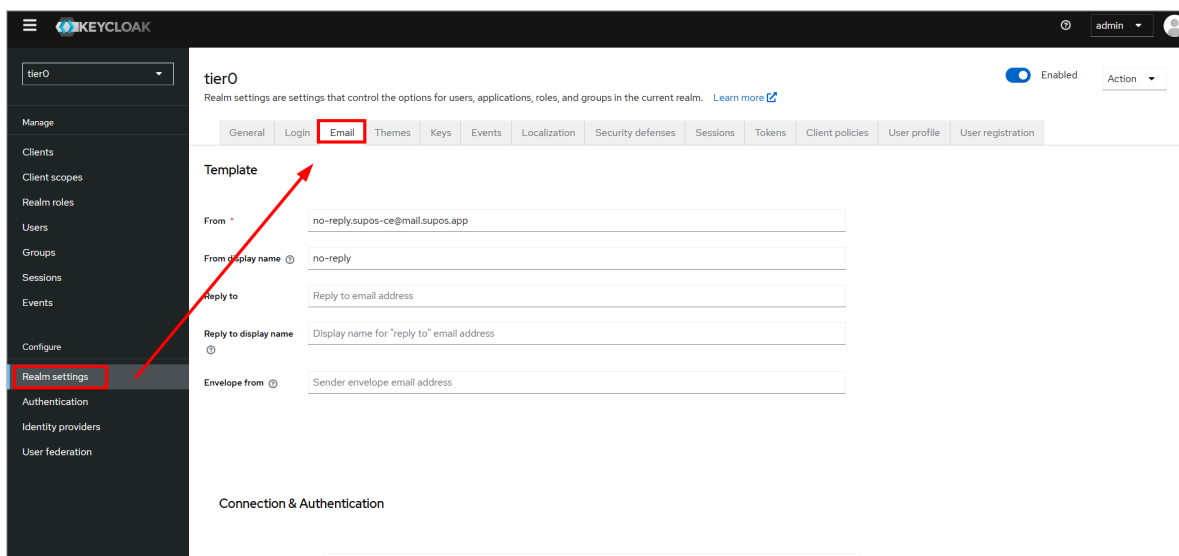
注册用户

用户注册需填写邮箱，需先配置邮件服务器以发送验证邮件。

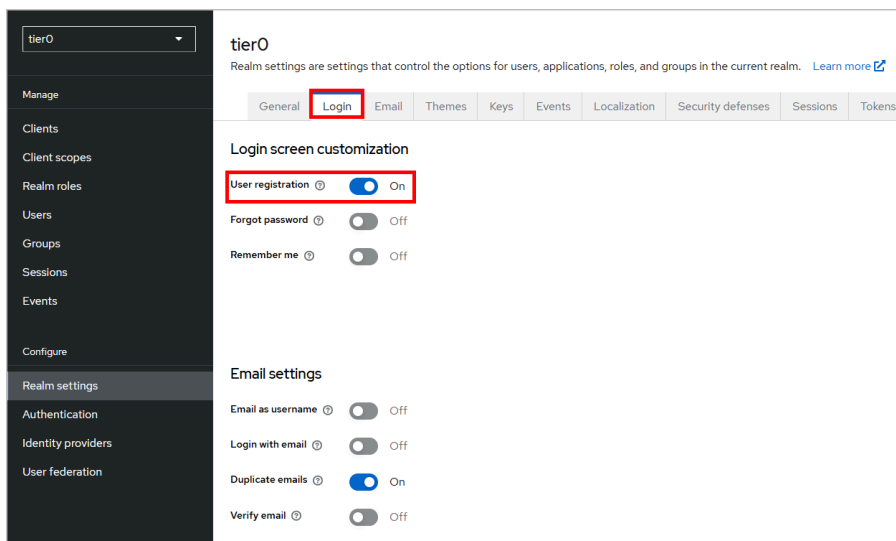
1. 选择 系统管理 > 身份与访问管理，使用账号密码（admin/tier0）登录 Keycloak。
2. 将 Realm 切换至 tier0，选择 Realm settings。



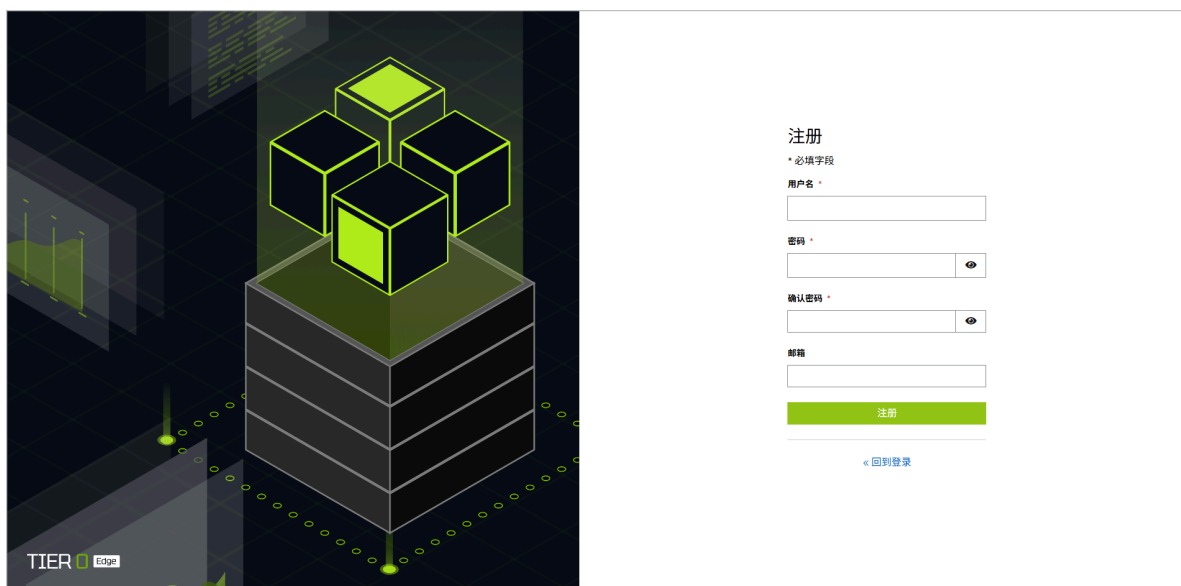
3. 单击 Email，填写邮件模板及服务器信息。



4. 保存设置后，单击 Login，启用 User registration 功能。



5. 进入平台登录页面，单击 **Register**。
6. 填写相关信息，申请用户账号。



开放数据

如何在 Tier0 中访问数据

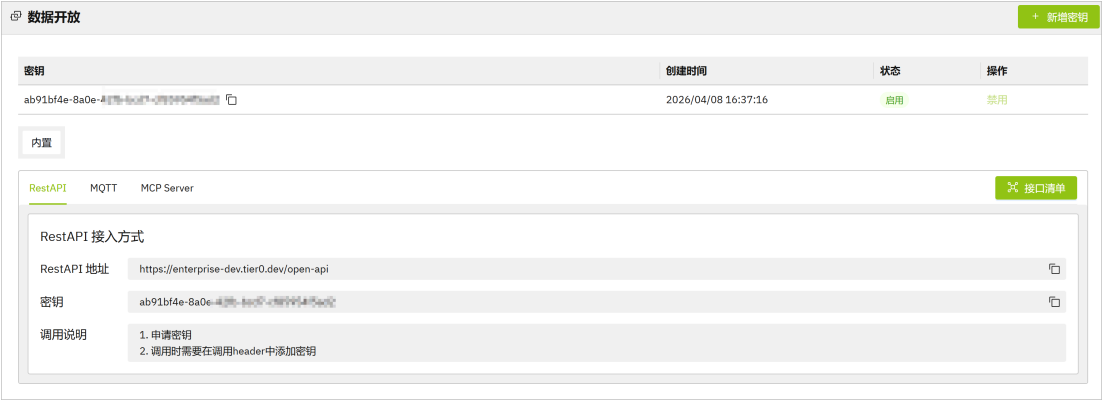
Tier0 提供 3 种数据订阅与调用方式：API、MQTT 和 MCP。

API

如需通过 API 访问数据，需要使用平台中提供的 URL 和密钥。

1. 登录 Tier0，选择 **系统管理** > **统一数据开放**。

2. 点击 **新增密钥** 创建密钥。



3. 使用底部显示的 URL 和创建的密钥，从 Tier0 获取数据。

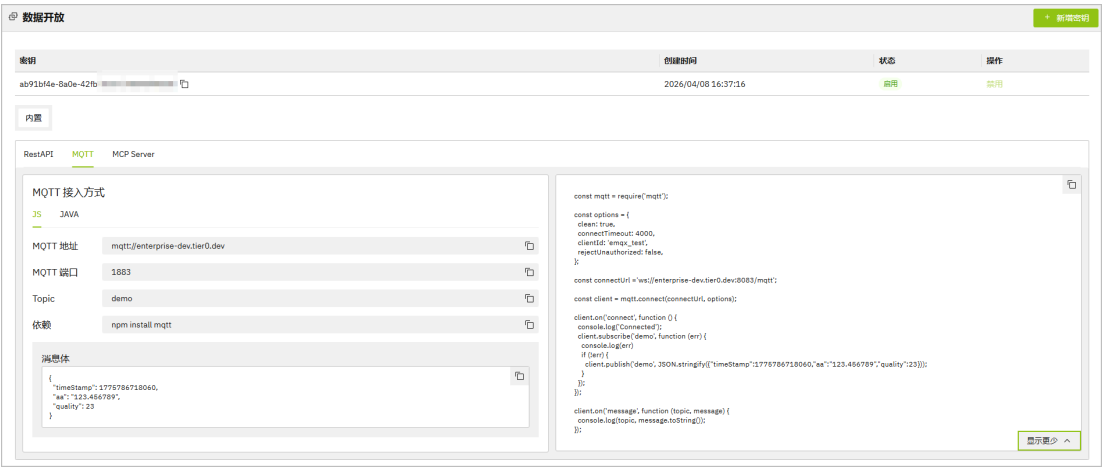
① 信息

点击 **接口清单** 可查看 API 文档。

MQTT

你可以使用 MQTTX、MQTT Explorer 等第三方客户端，或通过自定义代码，基于 MQTT 订阅数据。

1. 登录 Tier0，选择 **系统管理 > 统一数据开放**。
2. 点击 **MQTT** 查看 MQTT 连接信息，包括 broker 地址、端口和主题（topic）。



① 信息

Tier0 提供了 JavaScript 和 Java 两种脚本示例，帮助你快速上手。

MCP

Tier0 支持你通过 MCP 服务器 LLM 从平台获取数据。

- 1. 登录 Tier0，选择 系统管理 > 统一数据开放。
- 2. 点击 MCP Server 查看 MCP 连接详情和示例。



① 信息

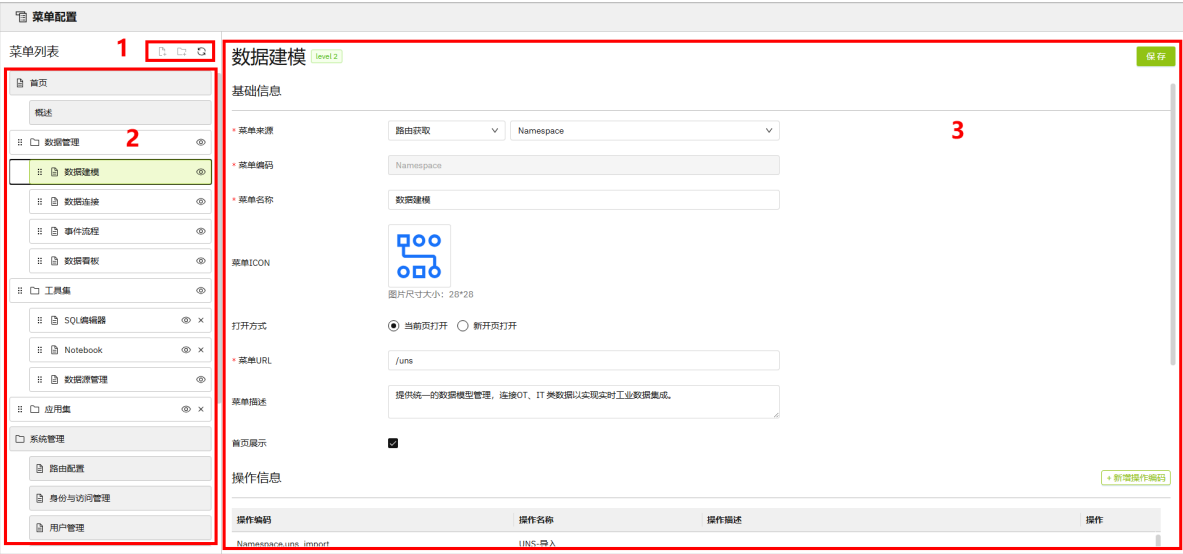
- 此处以 Cline 插件的使用方式作为示例。
- 所需的 URL 和密钥可在 RestAPI 页签中获取。
- 你可以在 RestAPI 页签中点击 接口清单，并复制其地址作为脚本中的 SUPOS_OPEN_PATH，以帮助 AI 理解。

菜单配置

平台支持以菜单形式对功能模块进行基础配置，包括模块位置、可见性及详细信息设置。同时，该功能可作为应用管理器，你可在平台上添加并配置自定义Web应用。

新增菜单

- 1. 登录平台后，选择 系统管理 > 菜单配置。



序号	名称	说明
1	操作	按文件夹/文件结构新增菜单，刷新菜单列表。
2	列表	展示已有的结构化菜单。
3	信息	展示所选菜单的详细信息。

2. 单击顶部的  图标，新增菜单文件夹。
3. 输入文件夹信息后，单击 保存。

新

level 1

保存

基础信息

菜单编码

菜单名称

菜单ICON

+

图片尺寸大小: 28*28

菜单描述

首页展示

☒

① 信息

菜单编码不可重复。

4. 选中已创建的文件夹，单击  图标，新增菜单文件。
5. 输入菜单信息后，单击 保存。

新level 2

保存

基础信息

菜单来源

手工

菜单编码

菜单名称

菜单ICON

+

图片尺寸大小: 28*28

打开方式

当前页打开

新开页打开

菜单URL

菜单描述

首页展示

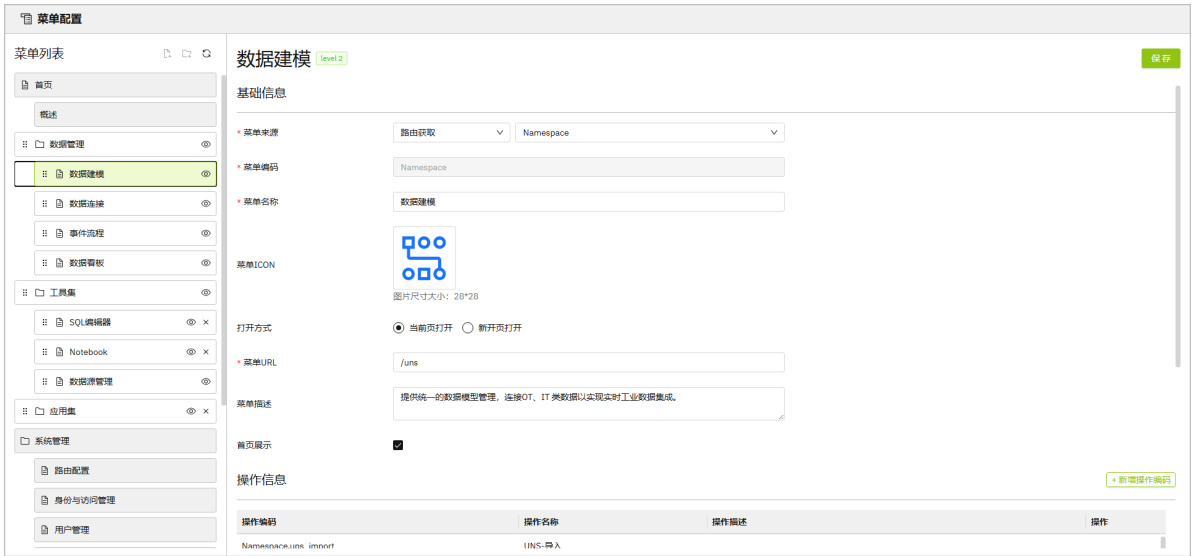
☒

参数	说明
菜单来源	选择菜单来源，支持手动新增或从已有选项中选择。
菜单编码	菜单的唯一标识。
打开方式	在当前页面直接打开菜单，或在新页面打开。
菜单URL	新增功能模块（菜单）的访问地址。
首页展示	是否在首页展示该菜单。

6. （可选）若设置为首页展示，返回 首页 即可查看新增的菜单。

编辑现有菜单

1. 登录平台后，选择 系统管理 > 菜单配置。
2. 从列表中选中目标菜单，在右侧编辑其信息后，单击 保存。



⚠ 注意

平台默认提供的原始菜单，请勿修改其菜单来源、地址及操作编码，否则可能导致模块无法正常使用。

管理菜单位置及可见性

- 在左侧列表中，拖拽菜单可调整其在文件夹/文件结构中的位置，调整后会自动同步至首页展示顺序。
- 单击菜单旁的  图标，可隐藏该菜单（首页不再显示）。
- 单击  图标，可删除菜单。

⚠ 注意

- 删除文件夹会同时删除其下所有菜单。
- 菜单删除后无法恢复，请谨慎操作。

术语表

核心术语及定义

术语	定义
Tier0	一款基于统一命名空间（UNS）方法论构建的开源工业数据集成平台，用于工业数据的管理、处理与优化。
UNS（统一命名空间）	一种数据管理方式，集中整合各类设备与系统的数据，提供结构化的统一框架，避免数据孤岛产生。
Node-RED	一款开源自动化工具，采用流式编程，帮助用户快速连接设备、API 与在线服务。
Grafana	一款开源监控与数据可视化平台，支持创建仪表盘，实时展示来自多数据源的数据。
MQTT	一种轻量级消息传输协议，适用于小型传感器与移动设备，专为高延迟或不稳定网络优化，是物联网（IoT）通信的常用协议。
JSON	JavaScript 对象表示法（JavaScript Object Notation），一种轻量级数据交换格式，易于人类读写，且便于机器解析和生成。
TimescaleDB	一款基于 PostgreSQL 构建的开源时序数据库，专为处理大规模时序数据设计，尤其适用于监控系统与物联网应用。
PostgreSQL	一款高级开源关系型数据库管理系统，采用 SQL 语言查询，具备高性能与高可靠性。
API（应用程序编程接口）	一组协议与工具集，用于不同软件应用之间的通信与交互。
Topic（主题）	Tier0 系统中的数据唯一标识，用于对多源数据进行分类整理，方便快速访问与管理。
OEE（设备综合效率）	制造业中评估设备效率与有效性的核心指标，综合考量设备的可用性、性能与产品质量。
AGV（自动导引车）	用于制造业或仓储环境的自动化运输设备，可实现物料自动搬运，减少物流环节的人工干预。
REST API（REST 接口）	一种 Web 服务接口，通过 HTTP 协议通信，支持 GET、POST、PUT、DELETE 等标准方法访问资源。
Time-Series Data（时序数据）	按固定时间间隔采集记录的数据，例如设备运行时间、温度及其他性能指标数据。

术语	定义
JSON Payload (JSON 负载)	以 JSON 对象格式传输的数据，包含传感器读数、设备状态等需处理或存储的相关信息。
Authentication (身份验证)	验证用户或系统身份的过程，确保只有授权实体才能访问相关资源。
Event-Driven (事件驱动)	一种编程范式，程序的执行流程由事件（如用户操作、传感器数据、其他系统消息等）触发。
JSONB	PostgreSQL 中的一种数据类型，以二进制格式存储 JSON 数据，相比普通 JSON 字段，具备更高效的索引、更快的查询速度和更优的性能。